C++26-Oriented Quant Interview Master Resource: Novice to Intermediate

Purpose

This document is designed to take you from **novice C++** to a strong **intermediate C++ interview level** for quant developer and quant research roles.

The goal is not to memorize random facts. The goal is to build a mental model that lets you reason through unfamiliar C++ questions.

You should be able to answer questions about:

- Memory and object lifetime
- Pointers, references, and ownership
- RAII and resource management
- Copying and moving
- STL containers and algorithms
- Templates and `typename`
- Virtual functions and object-oriented C++
- Concurrency and synchronization
- Performance and cache locality
- Numerical C++ for quant work
- Order books, market data, CSV parsing, and trading systems

This is **not** a LeetCode guide. It focuses on C++ language and systems knowledge.

---

0. How to study this document

C++ is difficult because it combines high-level abstraction with low-level control. If you only memorize syntax, interviewers can quickly expose gaps. Instead, study each topic using this loop:

1. **Understand the mental model.**
2. **Read the code example.**
3. **Predict what happens.**
4. **Explain why it happens.**
5. **Answer the interview question out loud.**
6. **Modify the example and recompile it.**

For every interview question, answer using this structure:

1. **Definition**: What is it?

2. **Mechanism**: How does it work?
3. **Example**: Show a simple example.
4. **Pitfall**: What can go wrong?
5. **Tradeoff**: When would you use or avoid it?
6. **Production angle**: Why does it matter in real systems?

Example:

> A reference is an alias for an existing object. It must be initialized and cannot normally be null or reseated. I use references for required function arguments, such as `void update(Order& order)`. If the argument is optional, I use a pointer or `std::optional`. References are non-owning, so they do not control object lifetime. The main pitfall is dangling references if the original object is destroyed.

This style makes you sound precise and practical.

---

1. The big picture: what C++ is trying to give you

1.1 C++ is a systems language with abstractions

C++ gives you:

- Direct memory control
- Deterministic object destruction
- Zero-cost abstractions when used well
- High-performance standard library containers and algorithms
- Generic programming with templates
- Low-level concurrency primitives
- Ability to build systems close to hardware

This is why C++ is common in:

- Trading systems
- Market data infrastructure
- Pricing libraries
- Risk engines
- Simulation engines
- Game engines
- Databases
- Compilers
- Embedded systems
- High-performance computing

The price is complexity. In C++, you must understand what owns memory, when objects die, and whether references/pointers remain valid.

1.2 The most important C++ question

For almost every C++ object, ask:

> Who owns this object, where does it live, and when is it destroyed?

If you can answer that, many C++ bugs become easier to reason about.

Consider:

```
std::vector<int> make_values() {
    std::vector<int> v = {1, 2, 3};
    return v;
}
```

Questions:

- Where does `v` live?
- Where do the vector elements live?
- What happens when `v` is returned?
- Who destroys the elements?

Answer:

- The local vector object `v` has automatic storage duration inside the function.
- Its elements are usually in heap-allocated memory owned by the vector.
- Returning by value is efficient due to copy elision or move semantics.
- The returned vector owns the elements and destroys them automatically.

This is the C++ mindset.

1.3 What quant interviews care about

For quant developer/researcher C++ interviews, interviewers care about whether you can write correct, efficient code in a performance-sensitive environment.

They may ask about:

- Why `std::vector` is usually faster than `std::list`
- What happens when a vector reallocates
- Why a base class destructor should be virtual
- Why raw owning pointers are dangerous
- How a hash table works
- How to prevent deadlock
- How to represent prices safely

- How to design an order book
- How to avoid unnecessary allocations in a hot path
- How to write exception-safe code

They are testing your ability to reason, not just recite definitions.

---

2. Basic C++ program structure

2.1 Minimal program

```
#include <iostream>

int main() {
    std::cout << "Hello, C++\n";
    return 0;
}
```

What each part means:

```
#include <iostream>
```

This includes declarations for input/output utilities such as `std::cout`.

```
int main()
```

`main` is the program entry point. It returns an integer status code to the operating system.

```
std::cout << "Hello, C++\n";
```

`std::cout` prints to standard output. `std::` means the name is inside the standard library namespace.

2.2 Namespaces

A namespace groups names to avoid collisions.

```
namespace trading {
    struct Order {};
}
```

```
trading::Order order;
```

Avoid this in headers:

```
using namespace std;
```

Why?

Because it pollutes every file that includes your header and can create name conflicts.

In `.cpp` files, limited use is less dangerous, but many professional teams still avoid it.

2.3 Header and source files

C++ often separates declarations and implementations.

Header file:

```
// order.h
#pragma once

class Order {
public:
    explicit Order(int id);
    int id() const;

private:
    int id_;
};
```

Source file:

```
// order.cpp
#include "order.h"

Order::Order(int id) : id_(id) {}

int Order::id() const {
    return id_;
}
```

The header tells other files what exists. The source file defines how it works.

2.4 Build mental model

C++ compilation roughly has these stages:

1. Preprocessing: expands `#include` and macros.
2. Compilation: turns each `.cpp` into object code.
3. Linking: combines object files into an executable.

This matters because many C++ errors are either:

- **Compile errors**: syntax/type errors in one translation unit.
- **Link errors**: declarations exist but definitions are missing or duplicated.
- **Runtime errors**: program compiles but fails while running.
- **Undefined behavior**: program has no valid meaning, even if it appears to run.

2.5 Interview questions

Q: What is a header file used for?

Strong answer:

> A header file usually contains declarations that other translation units need: class declarations, function declarations, templates, constants, and inline functions. Source files contain definitions. Headers are included textually by the preprocessor, so they should be clean, guarded, and should not define non-inline global variables.

Follow-up: What is a translation unit?

Answer:

> A translation unit is a source file after preprocessing, meaning after all `#include` files and macros are expanded. The compiler compiles each translation unit separately, and the linker later combines them.

Follow-up: Why can templates be defined in headers?

Answer:

> Templates need their definitions visible at the point of instantiation because the compiler generates code for specific template arguments. That is why most template code lives in headers.

---

3. Types, variables, and initialization

3.1 C++ is statically typed

Every expression has a type known at compile time.

```
int n = 5;
double x = 3.14;
std::string s = "AAPL";
```

Types matter because they determine:

- Memory size
- Operations allowed
- Function overload selection
- Copy/move behavior
- Performance

3.2 Fundamental types

Common types:

```
bool flag = true;
char c = 'A';
int n = 42;
long long big = 1'000'000'000LL;
float f = 1.5f;
double d = 1.5;
```

For precise-size integers, use `<cstdint>`:

```
#include <cstdint>

std::int32_t x = 10;
std::int64_t y = 10000000000LL;
std::uint64_t id = 123;
```

In trading systems, precise-size integers are common for IDs, timestamps, and price ticks.

3.3 Signed vs unsigned

```
int a = -1;
unsigned int b = 1;
```

Unsigned values cannot represent negative numbers. Be careful mixing signed and unsigned:

```
std::vector<int> v = {1, 2, 3};
for (int i = 0; i < v.size(); ++i) { // warning: signed/unsigned comparison
}
```

`v.size()` returns `std::size_t`, an unsigned type.

Better:

```
for (std::size_t i = 0; i < v.size(); ++i) {
}
```

Or range loop:

```
for (int x : v) {
}
```

Interview point:

> Unsigned types are not automatically safer. They can underflow silently. Use them when the domain is naturally non-negative and you understand arithmetic behavior, but do not use unsigned merely to prevent negative values.

3.4 Initialization styles

C++ has several initialization forms:

```
int a = 5;      // copy initialization
int b(5);       // direct initialization
int c{5};       // brace initialization
```

Brace initialization is often preferred because it prevents narrowing conversions:

```
int x = 3.14;  // allowed, x becomes 3
int y{3.14};   // error: narrowing conversion
```

For novice-to-intermediate code, prefer braces when practical:

```
std::vector<int> values{1, 2, 3};
Order order{123, 100, 10125};
```

3.5 `auto`

`auto` asks the compiler to deduce the type.

```
auto x = 5;       // int
auto y = 3.14;    // double
auto s = std::string{"AAPL"};
```

Good uses:

```
auto it = prices.find("AAPL");
```

Potentially bad use:

```
auto x = 0; // maybe unclear if int was intended
```

Important: `auto` often drops references and top-level const.

```
int n = 5;
int& r = n;
auto x = r;   // x is int, a copy
auto& y = r;  // y is int&, reference
```

3.6 Interview questions

Q: Why use brace initialization?

Answer:

> Brace initialization gives a uniform syntax and prevents narrowing conversions. For example, `int x{3.14}` is rejected, while `int x = 3.14` silently truncates.

Q: Is `auto` good or bad?

Answer:

> `auto` is useful when the type is obvious or verbose, such as iterators or template-heavy code. But it should not hide important information. Also, plain `auto` drops references, so use `auto&`, `const auto&`, or `decltype(auto)` when preserving reference semantics matters.

---

4. Storage duration, scope, and lifetime

This is one of the most important C++ topics.

4.1 Scope vs lifetime

Scope is where a name can be used.

Lifetime is when the object exists.

```
void f() {
    int x = 5; // x is in scope from here to end of block
} // x lifetime ends here
```

Usually local variable scope and lifetime align, but not always.

4.2 Automatic storage duration

Local variables usually have automatic storage duration.

```
void f() {
    int x = 5;
    std::string s = "hello";
} // x and s destroyed here
```

This is often called “stack allocation,” though the standard does not require a hardware stack.

4.3 Dynamic storage duration

Objects created dynamically live until explicitly destroyed or until an owner destroys them.

Bad old style:

```
int* p = new int(5);
delete p;
```

Modern style:

```
auto p = std::make_unique<int>(5);
```

The `unique_ptr` automatically deletes the object.

4.4 Static storage duration

Global and static variables live for the entire program.

```
int global_value = 10;

void f() {
    static int count = 0;
    ++count;
}
```

`count` is initialized once and retains its value across calls.

Be careful with global state in tests and multi-threaded programs.

4.5 Dangling references and pointers

A pointer/reference dangles when it refers to an object whose lifetime has ended.

Bad:

```
int& bad_ref() {
    int x = 5;
    return x; // dangling reference
}
```

Bad:

```
int* bad_ptr() {
    int x = 5;
```

```
    return &x; // dangling pointer
}
```

Good:

```
int good_value() {
    int x = 5;
    return x; // returns a copy
}
```

Good:

```
std::unique_ptr<int> good_ptr() {
    return std::make_unique<int>(5);
}
```

4.6 Lifetime of temporaries

Temporary objects usually live until the end of the full expression.

```
const std::string& s = std::string("hello");
```

This is okay because binding a temporary to a const reference extends its lifetime to match the reference lifetime.

But this is dangerous:

```
std::string_view sv = std::string("hello"); // dangling after this statement
```

`string_view` is not a reference in the lifetime-extension sense. It does not own data and does not extend lifetime.

4.7 Interview questions

Q: What is object lifetime?

Answer:

> Object lifetime is the period during which an object exists and can be safely used. It begins after construction and ends when the destructor starts or storage is released. Using pointers

> or references after lifetime ends causes dangling references/pointers and usually undefined behavior.

Follow-up: What is a dangling reference?

Answer:

> A dangling reference refers to an object that no longer exists. For example, returning a reference to a local variable creates a dangling reference because the local variable is destroyed when the function returns.

Follow-up: Why is returning `std::vector<int>` by value okay?

Answer:

> Modern C++ efficiently returns objects by value using copy elision or move semantics. The returned object owns its resources, so there is no dangling reference.

---

5. Undefined behavior

5.1 What undefined behavior means

Undefined behavior means the C++ standard gives no meaning to the program. The compiler can assume it never happens.

This is very different from an exception.

Exception:

```
throw std::runtime_error("error");
```

Defined behavior: stack unwinds.

Undefined behavior:

```
int* p = nullptr;
*p = 5;
```

No guarantees.

5.2 Common sources of UB

Null dereference

```
int* p = nullptr;
std::cout << *p;
```

Out-of-bounds access

```
int a[3] = {1, 2, 3};
std::cout << a[10];
```

Dangling reference

```
const std::string& bad() {
    std::string s = "abc";
    return s;
}
```

Use after free

```
int* p = new int(5);
delete p;
std::cout << *p;
```

Data race

```
int x = 0;
// two threads both do ++x without synchronization
```

Signed integer overflow

```
int x = std::numeric_limits<int>::max();
++x;
```

5.3 Why UB is dangerous in optimized builds

The compiler optimizes under the assumption that undefined behavior does not occur. Therefore, UB can cause surprising behavior far away from the bug.

Example:

```
int f(int* p) {
    int x = *p;
    if (p == nullptr) return 0;
    return x;
}
```

The compiler may remove the null check because dereferencing `p` already assumes `p` is non-null.

5.4 Interview questions

Q: What is undefined behavior?

Answer:

> Undefined behavior means the C++ standard imposes no requirements on what happens. The program might crash, appear to work, or be optimized incorrectly. Common causes are out-of-bounds access, dangling references, null dereference, signed overflow, and data races.

Follow-up: Is UB always caught by the compiler or runtime?

Answer:

> No. UB often compiles and may appear to work. Tools like sanitizers help detect many UB cases, but not all.

Follow-up: How do you prevent UB?

Answer:

> Use RAII, standard containers, bounds-checked access when appropriate, smart pointers for ownership, clear lifetime rules, compiler warnings, sanitizers, and careful synchronization in multi-threaded code.

---

6. RAII and resource management

6.1 Why RAII exists

C++ does not have garbage collection by default. Instead, it has deterministic destruction.

RAII uses deterministic destruction to manage resources automatically.

RAII means:

- Acquire resource in constructor.
- Release resource in destructor.
- Let scope control cleanup.

6.2 Simple RAII example

```
class Trace {
public:
    Trace() {
        std::cout << "start\n";
    }

    ~Trace() {
        std::cout << "end\n";
    }
};

void f() {
    Trace t;
    std::cout << "inside\n";
}
```

Output:

```
start
inside
end
```

The destructor runs automatically.

6.3 RAII with memory

Instead of:

```
void bad() {
    int* p = new int(5);
    if (some_condition()) return; // leak
    delete p;
}
```

Use:

```
void good() {
    auto p = std::make_unique<int>(5);
    if (some_condition()) return; // no leak
}
```

`unique_ptr` destructor deletes the integer.

6.4 RAII with locks

Without RAII:

```
m.lock();
do_work();
m.unlock();
```

If `do_work()` throws, `unlock()` is skipped.

With RAII:

```
std::lock_guard<std::mutex> lock(m);
do_work();
```

The lock is released automatically.

6.5 RAII with files

Use standard library file streams:

```
std::ifstream file("data.csv");
if (!file) {
    throw std::runtime_error("failed to open file");
}
```

When `file` goes out of scope, it closes automatically.

6.6 Rule of Zero

If your class uses RAII members, you usually do not need custom destructor/copy/move logic.

```
class Portfolio {
public:
    void add(std::string symbol, double qty) {
        symbols_.push_back(std::move(symbol));
        quantities_.push_back(qty);
    }

private:
    std::vector<std::string> symbols_;
    std::vector<double> quantities_;
};
```

No destructor needed. `vector` and `string` clean themselves.

6.7 Rule of Five

If your class directly owns a raw resource, you must think about all five special member functions.

```
class RawBuffer {
public:
    explicit RawBuffer(std::size_t n)
        : n_(n), data_(new double[n]) {}

    ~RawBuffer() {
        delete[] data_;
    }

private:
    std::size_t n_;
    double* data_;
};
```

This class is broken if copied:

```
RawBuffer a(10);
RawBuffer b = a; // default copy copies pointer only
```

Now both objects point to the same memory. Both destructors call `delete[]` on the same pointer. Double delete.

Fix options:

1. Delete copying.

2. Implement deep copy.
3. Prefer `std::vector<double>` and follow Rule of Zero.

6.8 Interview questions

Q: What is RAII?

Answer:

> RAII is a C++ idiom where resource management is tied to object lifetime. A constructor acquires a resource and the destructor releases it. This gives automatic cleanup during normal scope exit and exception unwinding.

Follow-up: Give examples.

Answer:

> `std::vector` manages heap memory, `std::unique_ptr` manages dynamically allocated objects, `std::lock_guard` manages a mutex lock, and `std::ifstream` manages a file handle.

Follow-up: What problem does RAII solve?

Answer:

> It prevents leaks and inconsistent cleanup paths. Without RAII, every return path and exception path must manually release resources. With RAII, destruction handles cleanup automatically.

Q: What is the Rule of Zero?

Answer:

> The Rule of Zero says that most classes should avoid manually defining destructors, copy constructors, move constructors, and assignment operators. Instead, they should store RAII members like `vector`, `string`, and smart pointers that manage their own resources.

Q: What is the Rule of Five?

Answer:

> If a class directly manages a resource and needs one special member function, it likely needs destructor, copy constructor, copy assignment, move constructor, and move assignment. This is because copying, moving, and destroying all affect resource ownership.

---

7. Pointers, references, and ownership

7.1 Why this matters

Most serious C++ bugs involve bad ownership or invalid lifetime:

- Use-after-free
- Double delete
- Dangling pointer
- Dangling reference
- Memory leak
- Null dereference

You need to distinguish:

- Accessing an object
- Owning an object
- Sharing ownership
- Observing without ownership

7.2 References

A reference is an alias.

```
int x = 10;
int& r = x;
r = 20;
std::cout << x; // 20
```

`r` is another name for `x`.

Important properties:

- Must be initialized.
- Cannot be reseated.
- Usually cannot be null in valid code.
- Does not own the object.

Function example:

```
void apply_fee(double& price) {
    price *= 1.001;
}
```

This modifies the caller's variable.

7.3 Const references

Use `const&` for read-only access to larger objects.

```
void print_symbol(const std::string& symbol) {
    std::cout << symbol << '\n';
}
```

This avoids copying the string while preventing mutation.

For small types, pass by value:

```
void set_price(double price); // good
```

Do not write:

```
void set_price(const double& price); // usually unnecessary
```

A `double` is cheap to copy.

7.4 Pointers

A pointer stores an address.

```
int x = 10;
int* p = &x;
*p = 20;
```

Properties:

- Can be null.
- Can be reassigned.
- Can support pointer arithmetic.
- Does not necessarily own memory.

Function example:

```
void maybe_update(Order* order) {
    if (order) {
```

```
        order->quantity += 10;
    }
}
```

The pointer expresses optionality.

7.5 Dot vs arrow

Object or reference:

```
order.quantity = 100;
```

Pointer:

```
order_ptr->quantity = 100;
```

`p->x` is shorthand for `(*p).x`.

7.6 Ownership categories

Value ownership

```
std::vector<Order> orders;
```

The vector owns the `Order` objects directly.

Exclusive heap ownership

```
std::unique_ptr<Model> model;
```

Exactly one owner.

Shared ownership

```
std::shared_ptr<Model> model;
```

Multiple owners; object dies when last owner dies.

Non-owning observation

```
Model* model;      // optional observer
Model& model_ref;  // required observer
```

7.7 `unique_ptr`

Use `unique_ptr` for exclusive ownership of heap objects.

```
auto model = std::make_unique<Model>();
```

Transfer ownership:

```
void set_model(std::unique_ptr<Model> model) {
    model_ = std::move(model);
}
```

Calling:

```
auto m = std::make_unique<Model>();
strategy.set_model(std::move(m));
```

After `std::move(m)`, `m` no longer owns the object.

7.8 `shared_ptr`

Use only when shared lifetime ownership is truly needed.

```
auto model = std::make_shared<Model>();
std::shared_ptr<Model> another = model;
```

Reference count is now 2.

Costs:

- Control block allocation
- Reference count increments/decrements
- Atomic operations in many implementations
- Possible cycles
- Less clear ownership

7.9 `weak_ptr`

Use `weak_ptr` to observe a `shared_ptr` object without extending lifetime.

```
std::weak_ptr<Model> weak = model;

if (auto locked = weak.lock()) {
    locked->run();
}
```

`lock()` returns a `shared_ptr` if the object still exists.

7.10 Why `vector<T&>` is invalid

This is common in interviews.

```
std::vector<int&> refs; // invalid
```

Why?

A reference is not a regular object. Containers need elements that can be assigned, moved, destroyed, and stored. A reference cannot be reseated.

Alternatives:

```
std::vector<int*> ptrs;
std::vector<std::reference_wrapper<int>> refs;
std::vector<std::unique_ptr<int>> owned;
std::vector<int> values;
```

Each has different semantics.

7.11 Interview questions

Q: Pointer vs reference?

Answer:

> A reference is an alias for an existing object. It must be initialized and cannot be reseated. A pointer stores an address, can be null, and can be reassigned. I use references for required objects and pointers when nullability or reseating is meaningful.

Follow-up: Can a reference be null?

Answer:

> In valid C++, a reference should refer to a real object. You can create invalid situations by dereferencing a null pointer and binding to a reference, but that is undefined behavior. Semantically, references should be treated as non-null.

Q: When should primitives be passed by reference?

Answer:

> Usually only when the function needs to modify the caller's variable. For read-only use, small primitives like `int`, `double`, and pointers should usually be passed by value because copying them is cheap and clearer.

Q: When use `unique_ptr` vs `shared_ptr`?

Answer:

> Use `unique_ptr` for exclusive ownership. It is simple and cheap. Use `shared_ptr` only when multiple independent owners must share lifetime. If one object only observes another, use raw pointer, reference, or `weak_ptr` depending on ownership context.

Q: Why is raw `new` discouraged?

Answer:

> Raw `new` creates manual ownership. If an exception or early return happens, memory may leak. Smart pointers and containers express ownership and clean up automatically.

Q: Is a raw pointer always bad?

Answer:

> No. Raw pointers are fine for non-owning observation, especially when null is meaningful. What is dangerous is raw owning pointers without clear lifetime management.

---

8. `const`, immutability, and API clarity

8.1 Why `const` matters

`const` is not just compiler decoration. It communicates intent.

```
void print_order(const Order& order);
```

This says the function will not modify the order.

Benefits:

- Documents API behavior.
- Prevents accidental mutation.
- Allows passing const objects.
- Helps reasoning in large systems.

8.2 Const variables

```
const double risk_limit = 1'000'000.0;
```

Cannot modify after initialization.

8.3 Pointer const combinations

```
const int* p;       // pointer to const int
int* const q = &x;  // const pointer to int
const int* const r; // const pointer to const int
```

Mental model:

- `const int* p`: cannot modify `*p`; can change `p`.
- `int* const p`: can modify `*p`; cannot change `p`.
- `const int* const p`: can modify neither through this name.

8.4 Const member functions

```
class OrderBook {
public:
    double mid() const {
        return 0.5 * (best_bid_ + best_ask_);
    }

    void update(double bid, double ask) {
        best_bid_ = bid;
        best_ask_ = ask;
    }
```

```
private:
    double best_bid_ = 0.0;
    double best_ask_ = 0.0;
};
```

`mid()` can be called on a const `OrderBook`.

```
const OrderBook book;
book.mid();    // allowed
// book.update(); // not allowed
```

8.5 Logical vs physical constness

A const member function promises not to modify logical state.

Sometimes internal caches are allowed to change:

```
class Model {
public:
    double value() const {
        if (!cached_) {
            cached_value_ = expensive_compute();
            cached_ = true;
        }
        return cached_value_;
    }

private:
    double expensive_compute() const { return 42.0; }

    mutable bool cached_ = false;
    mutable double cached_value_ = 0.0;
};
```

`mutable` allows modification inside const functions. Use sparingly.

8.6 `constexpr`

`constexpr` means something can be evaluated at compile time when possible.

```
constexpr double square(double x) {
    return x * x;
}
```

```
constexpr double y = square(3.0);
```

Use for:

- Compile-time constants
- Small pure functions
- Fixed configuration

8.7 Interview questions

Q: What is const-correctness?

Answer:

> Const-correctness means marking objects, function parameters, and member functions as `const` when they should not modify logical state. It improves clarity and allows the compiler to catch accidental mutation.

Follow-up: What is a const member function?

Answer:

> A const member function promises not to modify the object's logical state. It can be called on const objects. Inside such a function, `this` behaves like a pointer to const.

Q: What is `mutable`?

Answer:

> `mutable` allows a member to be modified even inside a const member function. It is usually used for caches, mutexes, or instrumentation that does not change logical state.

Q: `const int*` vs `int* const`?

Answer:

> `const int*` is a pointer to const int; you cannot modify the int through the pointer, but you can point elsewhere. `int* const` is a const pointer to int; you can modify the int, but cannot change the pointer itself.

---

9. Classes, constructors, destructors, and encapsulation

9.1 What a class is

A class groups data and functions.

```
class Order {
public:
    Order(int id, double qty, double price)
        : id_(id), quantity_(qty), price_(price) {}

    double notional() const {
        return quantity_ * price_;
    }

private:
    int id_;
    double quantity_;
    double price_;
};
```

`public` members are part of the interface. `private` members are implementation details.

9.2 Encapsulation

Encapsulation means hiding internal representation and exposing safe operations.

Bad:

```
struct Position {
    double quantity;
    double avg_price;
};
```

This may be fine for simple data, but there are no invariants.

Better if invariants matter:

```
class Position {
public:
```

```
    void add_fill(double qty, double price) {
        // update quantity and average price consistently
    }

    double quantity() const { return quantity_; }
    double avg_price() const { return avg_price_; }

private:
    double quantity_ = 0.0;
    double avg_price_ = 0.0;
};
```

Now you control updates.

9.3 Constructor initialization lists

Prefer:

```
class Order {
public:
    Order(int id, std::string symbol)
        : id_(id), symbol_(std::move(symbol)) {}

private:
    int id_;
    std::string symbol_;
};
```

Instead of:

```
Order(int id, std::string symbol) {
    id_ = id;
    symbol_ = std::move(symbol);
}
```

Why?

The initialization list constructs members directly. Assignment in the body may default-construct first, then assign.

9.4 Initialization order

Members initialize in declaration order.

```
class Example {
public:
    Example() : y_(x_), x_(10) {}

private:
    int x_;
    int y_;
};
```

Despite list order, `x_` initializes before `y_` because `x_` is declared first.

Keep initialization list order the same as declaration order.

9.5 `explicit` constructors

Single-argument constructors can create implicit conversions.

```
class Price {
public:
    Price(double value) : value_(value) {}
private:
    double value_;
};

void trade(Price p);
trade(100.0); // allowed
```

This may be surprising. Prefer:

```
class Price {
public:
    explicit Price(double value) : value_(value) {}
private:
    double value_;
};
```

Now:

```
trade(Price{100.0});
```

9.6 Destructors

A destructor runs when an object dies.

```
class Connection {
public:
    ~Connection() {
        close();
    }
};
```

Destructors should generally not throw.

9.7 Struct vs class

In C++:

```
struct S {
    int x;
};
```

Members are public by default.

```
class C {
    int x;
};
```

Members are private by default.

Use `struct` for simple passive data. Use `class` when you need invariants and encapsulation.

9.8 Interview questions

Q: Why use initialization lists?

Answer:

> Initialization lists construct members directly. Assignment in the constructor body may first default-construct members and then assign. Initialization lists are required for references, const members, and members without default constructors.

Q: What is `explicit`?

Answer:

> `explicit` prevents implicit conversions through constructors. It is especially useful for domain types like `Price`, `Quantity`, or `Volatility`, where accidental conversion from a raw number could hide bugs.

Q: Struct vs class?

Answer:

> The main language difference is default access: struct members are public, class members are private. By convention, structs are often used for simple data aggregates, while classes are used when invariants and encapsulation matter.

---

10. Copying, moving, and value categories

This is one of the most important intermediate C++ topics.

10.1 Why move semantics exist

Some objects own expensive resources.

Example:

```
std::vector<int> v(1'000'000);
```

Copying this vector means allocating new memory and copying one million integers.

Moving can simply transfer the internal pointer from one vector to another.

10.2 Copying

```
std::vector<int> a = {1, 2, 3};
std::vector<int> b = a;
```

After copy:

- `a` owns its own buffer.
- `b` owns a separate buffer.

- Mutating one does not affect the other.

10.3 Moving

```
std::vector<int> a = {1, 2, 3};
std::vector<int> b = std::move(a);
```

After move:

- `b` owns the original buffer.
- `a` is valid but unspecified.
- You can destroy `a` or assign to it.

Do not rely on `a` being empty unless documented.

10.4 `std::move`

`std::move` does not move. It casts.

```
std::move(a)
```

means:

> Treat `a` as an expiring object so move operations may be used.

The actual movement happens in the move constructor or move assignment operator.

10.5 Lvalues and rvalues

Simple model:

- Lvalue: has identity; can be named or addressed.
- Rvalue: temporary or expiring value.

```
int x = 5;   // x is an lvalue
int y = x+1; // x+1 is an rvalue/prvalue
```

Named variables are lvalues, even if their type is rvalue reference.

```
void f(std::string&& s) {
    std::string a = s;            // copy, because s is an lvalue expression
```

```
    std::string b = std::move(s); // move
}
```

10.6 Move constructor

```
class Buffer {
public:
    Buffer(Buffer&& other) noexcept
        : data_(std::move(other.data_)) {}

private:
    std::vector<double> data_;
};
```

For standard library members, default move is often enough:

```
class Buffer {
public:
    Buffer(Buffer&&) noexcept = default;
    Buffer& operator=(Buffer&&) noexcept = default;

private:
    std::vector<double> data_;
};
```

10.7 Copy elision

Returning by value is efficient:

```
std::vector<int> make_values() {
    std::vector<int> v = {1, 2, 3};
    return v;
}
```

The compiler can construct the returned object directly in the caller.

Avoid this:

```
return std::move(v);
```

It may inhibit copy elision.

10.8 Pass-by-value-and-move

Useful for constructors that store a copy.

```
class Strategy {
public:
    explicit Strategy(std::string name)
        : name_(std::move(name)) {}

private:
    std::string name_;
};
```

If caller passes an lvalue, it copies into `name`, then moves into `name_`. If caller passes an rvalue, it moves.

This is clean and often efficient.

10.9 `noexcept` move

Standard containers may prefer copying over moving if move might throw.

```
class Order {
public:
    Order(Order&&) noexcept = default;
    Order& operator=(Order&&) noexcept = default;
};
```

Mark move operations `noexcept` when they cannot throw.

10.10 Interview questions

Q: What is the difference between copy and move?

Answer:

> Copying creates an independent duplicate. Moving transfers resources from one object to another, often avoiding allocation and deep copy. After a move, the source object remains valid but has unspecified value.

Q: What does `std::move` do?

Answer:

> It casts an expression to an rvalue reference. It does not move anything by itself. It enables move constructors or move assignment operators to be selected.

Follow-up: Why do we need `std::move` inside a move constructor?

Answer:

> Because named variables are lvalues even if their type is rvalue reference. To move from a member of the source object, we must cast it with `std::move`.

Q: What is copy elision?

Answer:

> Copy elision is a compiler optimization where copies or moves are omitted by constructing an object directly in its final location. Modern C++ guarantees copy elision in some cases, making return-by-value efficient.

Q: Why should move constructors be `noexcept`?

Answer:

> Containers like `std::vector` can use move operations during reallocation only if moving is safe from throwing, otherwise they may copy to preserve exception safety.

---

11. Object-oriented C++ and virtual functions

11.1 Why OOP exists

Object-oriented programming helps model interfaces and polymorphic behavior.

Example:

```
class Instrument {
public:
    virtual ~Instrument() = default;
    virtual double price() const = 0;
};
```

Different instruments can implement pricing differently:

```
class EuropeanCall : public Instrument {
public:
    double price() const override {
        return 1.23;
    }
};

class Bond : public Instrument {
public:
    double price() const override {
        return 99.5;
    }
};
```

A function can work with any instrument:

```
void print_price(const Instrument& inst) {
    std::cout << inst.price() << '\n';
}
```

11.2 Virtual functions

A virtual function is dispatched based on runtime type.

```
Instrument& inst = call;
inst.price(); // calls EuropeanCall::price()
```

Without `virtual`, the function call is resolved based on static type.

11.3 Vtable mental model

Most compilers implement virtual functions using:

- A hidden pointer in each polymorphic object: vptr
- A table of function pointers for each class: vtable

A virtual call does roughly:

1. Load object's vptr.
2. Find function pointer in vtable.
3. Call through function pointer.

Costs:

- Extra pointer in object
- Indirect call
- Harder to inline
- Possible branch prediction/cache effects

But virtual functions are often perfectly fine outside ultra-hot loops.

11.4 Pure virtual functions and abstract classes

```
class Strategy {
public:
    virtual ~Strategy() = default;
    virtual void on_tick(const Tick& tick) = 0;
};
```

`= 0` makes the function pure virtual. The class is abstract and cannot be instantiated directly.

11.5 Virtual destructor

Always use a virtual destructor in polymorphic base classes.

```
class Base {
public:
    virtual ~Base() = default;
};
```

Why?

```
Base* p = new Derived();
delete p;
```

If `Base` destructor is not virtual, deleting through `Base*` is undefined behavior.

11.6 Object slicing

```
void f(Instrument inst); // pass by value

EuropeanCall call;
f(call); // slices derived part
```

The `EuropeanCall` part is lost because only the base subobject is copied.

Fix:

```
void f(const Instrument& inst);
```

11.7 Inheritance vs composition

Inheritance means “is-a.”

```
class EquityOption : public Instrument {};
```

Composition means “has-a.”

```
class Strategy {
    RiskModel risk_model_;
};
```

Prefer composition unless true substitutability exists.

11.8 Static polymorphism

Templates can provide compile-time polymorphism.

```
template <typename Strategy>
void run(Strategy& strategy, const Tick& tick) {
    strategy.on_tick(tick);
}
```

Benefits:

- Can inline
- No virtual dispatch

Costs:

- More templates
- Longer compile times
- Potential code bloat

11.9 Interview questions

Q: What is a virtual function?

Answer:

> A virtual function allows runtime polymorphism. When called through a base pointer or reference, the derived class override is selected based on the object's dynamic type.

Follow-up: How is it implemented?

Answer:

> Usually through a vtable. Each polymorphic object has a hidden vptr pointing to a table of function pointers. A virtual call uses that table to call the correct override.

Follow-up: What is the cost?

Answer:

> The cost is usually one level of indirection, an extra vptr per object, and reduced inlining. It can matter in very hot loops but is acceptable in many designs.

Q: Why virtual destructor?

Answer:

> If a derived object is deleted through a base pointer, the base destructor must be virtual so the derived destructor runs. Without it, behavior is undefined.

Q: What is object slicing?

Answer:

> Object slicing happens when a derived object is copied into a base object by value, losing the derived part. Avoid by using references or pointers for polymorphic objects.

---

12. Templates and generic programming

12.1 Why templates exist

Templates let you write generic code without sacrificing type safety or performance.

Instead of writing:

```
int square_int(int x) { return x * x; }
double square_double(double x) { return x * x; }
```

Use:

```
template <typename T>
T square(T x) {
    return x * x;
}
```

The compiler generates versions for used types.

12.2 Function templates

```
template <typename T>
T max_value(T a, T b) {
    return a < b ? b : a;
}
```

Usage:

```
auto x = max_value(3, 5);
auto y = max_value(2.0, 4.0);
```

But this fails:

```
max_value(3, 4.0); // T cannot be both int and double
```

Fix:

```
template <typename T, typename U>
auto max_value(T a, U b) {
    return a < b ? b : a;
}
```

12.3 Class templates

```
template <typename T>
class Box {
```

```
public:
    explicit Box(T value) : value_(std::move(value)) {}
    const T& value() const { return value_; }

private:
    T value_;
};
```

Usage:

```
Box<int> a{5};
Box<std::string> b{"AAPL"};
```

12.4 `typename` vs `class`

In template parameter lists, these are equivalent:

```
template <typename T>
template <class T>
```

Most modern code uses `typename`.

12.5 Dependent names

This is a key intermediate concept.

```
template <typename Container>
void f(const Container& c) {
    typename Container::const_iterator it = c.begin();
}
```

`Container::const_iterator` depends on `Container`. Before instantiation, the compiler does not know whether it is a type or a static variable.

`typename` tells the compiler it is a type.

Without it, the compiler may not parse the code correctly.

12.6 `auto` in templates

Often you can avoid verbose dependent names:

```
template <typename Container>
void f(const Container& c) {
    auto it = c.begin();
}
```

But interviewers may still ask why `typename` exists.

12.7 Type traits

Type traits let you ask questions about types at compile time.

```
#include <type_traits>

template <typename T>
void process(T x) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "integer\n";
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "floating point\n";
    } else {
        std::cout << "other\n";
    }
}
```

`if constexpr` discards branches at compile time.

12.8 SFINAE high-level idea

SFINAE means substitution failure is not an error.

If a template overload cannot be formed for a type, the compiler removes it from overload resolution rather than failing immediately.

Older C++ used SFINAE heavily for constraints.

```
template <typename T, std::enable_if_t<std::is_integral_v<T>, int> = 0>
void only_integral(T x) {}
```

You do not need to master complex SFINAE initially, but you should understand the concept.

12.9 Concepts

C++20 concepts make constraints readable.

```
#include <concepts>

template <std::floating_point T>
T scale_vol(T price, T vol) {
    return price * vol;
}
```

Custom concept:

```
template <typename T>
concept HasPrice = requires(T x) {
    { x.price() } -> std::convertible_to<double>;
};

template <HasPrice T>
double get_price(const T& x) {
    return x.price();
}
```

12.10 Interview questions

Q: What is a template?

Answer:

> A template is a compile-time mechanism for generic programming. The compiler instantiates concrete versions of the template for the types used. Templates provide static polymorphism and can be optimized heavily.

Q: What is the difference between templates and virtual functions?

Answer:

> Templates provide compile-time polymorphism; virtual functions provide runtime polymorphism. Templates can be inlined and optimized for each type, but can increase compile time and binary size. Virtual functions are more flexible for runtime substitution but involve dynamic dispatch.

Q: When do you need `typename`?

Answer:

> You need `typename` before a dependent nested type name, such as `typename T::value_type`, because the compiler does not know whether `T::value_type` is a type until the template is instantiated.

Q: What are concepts?

Answer:

> Concepts are C++20 constraints on template parameters. They make requirements explicit and improve error messages compared with older SFINAE approaches.

---

13. STL containers: beginner-to-intermediate depth

13.1 Why containers matter

STL containers manage memory and provide common data structures.

The key interview skill is knowing:

- What the container stores
- Its complexity
- Its memory layout
- Iterator/reference invalidation
- When to use it

13.2 `std::vector`

A dynamic array with contiguous memory.

```
std::vector<int> v;
v.push_back(1);
v.push_back(2);
```

Internally, vector usually stores:

- Pointer to beginning
- Pointer to end of used elements

- Pointer to end of capacity

Size vs capacity

```
std::vector<int> v;
std::cout << v.size();     // number of elements
std::cout << v.capacity(); // allocated space
```

`reserve`

```
v.reserve(1000);
```

Allocates enough memory for at least 1000 elements but does not create them.

`resize`

```
v.resize(1000);
```

Changes size to 1000 and constructs elements.

Reallocation

If vector runs out of capacity, it allocates a larger buffer and moves/copies elements.

This invalidates pointers, references, and iterators to elements.

```
std::vector<int> v = {1, 2, 3};
int& r = v[0];
v.push_back(4); // may reallocate
// r may now dangle
```

When to use vector

Use vector by default for sequences.

Good for:

- Numeric arrays
- Time series
- Simulation paths
- Batches of orders/messages
- Most dynamic collections

13.3 `std::deque`

A double-ended queue.

```
std::deque<int> d;
d.push_back(1);
d.push_front(0);
```

Good for:

- Push/pop at both ends
- Queues
- Sliding windows

Not one contiguous block, so less cache-friendly than vector for scanning.

13.4 `std::list`

A doubly linked list.

```
std::list<int> l;
l.push_back(1);
l.push_back(2);
```

Pros:

- Stable iterators
- O(1) insert/erase given iterator

Cons:

- Poor cache locality
- Extra memory per node
- Allocation per node
- No random access

Use only when you truly need node stability or frequent middle insertion/removal.

13.5 `std::map`

Ordered key-value container, usually a balanced tree.

```
std::map<std::string, double> prices;
prices["AAPL"] = 190.0;
```

Properties:

- Sorted by key
- O(log n) lookup/insert/delete
- Stable iterators except erased elements

Use when ordering matters.

13.6 `std::unordered_map`

Hash table.

```
std::unordered_map<std::string, double> prices;
prices["AAPL"] = 190.0;
```

Properties:

- Average O(1) lookup/insert/delete
- No ordering
- Rehash can invalidate iterators
- Performance depends on hash quality and load factor

Use for fast lookup when order does not matter.

13.7 `std::set` and `std::unordered_set`

Like map/unordered_map but store only keys.

```
std::set<int> sorted_ids;
std::unordered_set<int> seen_ids;
```

13.8 `std::priority_queue`

Heap adaptor.

```
std::priority_queue<int> max_heap;
max_heap.push(10);
```

```
max_heap.push(5);
std::cout << max_heap.top(); // 10
```

Min heap:

```
std::priority_queue<int, std::vector<int>, std::greater<int>> min_heap;
```

Good for retrieving max/min repeatedly.

Not ideal when you need arbitrary deletion.

13.9 `std::optional`

Represents maybe-a-value.

```
std::optional<double> best_bid() {
    if (empty()) return std::nullopt;
    return bid;
}
```

Use instead of sentinel values when absence is meaningful.

13.10 `std::variant`

Type-safe union.

```
using Field = std::variant<int, double, std::string>;
Field f = 3.14;
```

Visit:

```
std::visit([](const auto& x) {
    std::cout << x << '\n';
}, f);
```

Useful for typed columns and messages.

13.11 `std::span`

Non-owning view over contiguous memory.

```
void normalize(std::span<double> xs) {
    for (double& x : xs) x *= 0.01;
}
```

Can accept vector, array, or raw buffer.

Does not own data.

13.12 `std::string_view`

Non-owning view over string data.

```
void parse_symbol(std::string_view symbol);
```

Avoids copying.

Danger:

```
std::string_view bad() {
    std::string s = "AAPL";
    return s; // dangling
}
```

13.13 Interview questions

Q: `vector` vs `list`?

Answer:

> `vector` stores elements contiguously, which gives excellent cache locality and fast iteration. `list` stores separate nodes linked by pointers, so insertion/removal given an iterator is O(1), but iteration is slower due to pointer chasing and poor locality. I use vector by default unless I specifically need stable iterators and frequent middle insertion/removal.

Q: `map` vs `unordered_map`?

Answer:

> `map` is ordered and gives O(log n) operations, usually using a balanced tree. `unordered_map` is hash-based and gives average O(1) operations but no ordering and possible worst-case O(n). Use map for sorted traversal/range queries and unordered_map for fast lookup.

Q: What invalidates vector iterators?

Answer:

> Reallocation invalidates all iterators, references, and pointers to elements. Insertions and erasures can also invalidate elements at or after the modification point.

Q: `reserve` vs `resize`?

Answer:

> `reserve` changes capacity but not size. `resize` changes the number of actual elements and constructs or destroys elements.

Q: What is `string_view`?

Answer:

> `string_view` is a non-owning view into character data. It avoids copies but can dangle if the underlying string does not outlive the view.

---

14. Iterators and invalidation

14.1 What is an iterator?

An iterator is an object that points into a container and allows traversal.

```
std::vector<int> v = {1, 2, 3};
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << '\n';
}
```

Range-based loop uses iterators internally:

```
for (int x : v) {
    std::cout << x << '\n';
}
```

14.2 Iterator categories

You do not need to memorize all details initially, but know the idea:

- Input iterator: read forward
- Forward iterator: multi-pass forward
- Bidirectional iterator: forward and backward
- Random-access iterator: jump by index-like operations
- Contiguous iterator: memory contiguous

`vector` has random-access/contiguous iterators. `list` has bidirectional iterators.

14.3 Invalidation

Invalidation means an iterator/reference/pointer no longer safely refers to an element.

Example:

```
std::vector<int> v = {1, 2, 3};
auto it = v.begin();
v.push_back(4); // may invalidate it
```

Using invalidated iterators is undefined behavior.

14.4 Common rules

Vector

- Reallocation invalidates all.
- Erase invalidates erased element and elements after it.
- Insert may invalidate all if reallocation happens.

List

- Insert does not invalidate existing iterators.
- Erase invalidates only erased iterator.

Map/set

- Insert does not invalidate existing iterators.
- Erase invalidates only erased element.

Unordered map/set

- Rehash invalidates iterators.
- Erase invalidates erased element.

14.5 Interview questions

Q: Why do iterator invalidation rules matter?

Answer:

> Because using an invalidated iterator or reference is undefined behavior. In systems code, it can lead to subtle memory bugs. For example, storing references into a vector is dangerous if the vector may grow and reallocate.

Q: How do you avoid vector invalidation?

Answer:

> Reserve capacity when possible, avoid storing long-lived references/pointers into vectors that may grow, use indices carefully, or choose containers with more stable iterators when needed.

---

15. Algorithms, lambdas, and idiomatic STL

15.1 Why algorithms matter

C++ standard algorithms make intent clear and reduce bugs.

Instead of manual loops for everything, use:

```
std::sort(v.begin(), v.end());
std::find(v.begin(), v.end(), x);
std::accumulate(v.begin(), v.end(), 0.0);
```

15.2 Sorting

```
std::sort(orders.begin(), orders.end(), [](const Order& a, const Order& b) {
    return a.price < b.price;
});
```

Comparator returns true if `a` should come before `b`.

15.3 `lower_bound`

Requires sorted range.

```
auto it = std::lower_bound(v.begin(), v.end(), target);
```

Finds first element not less than target.

15.4 `accumulate`

```
double sum = std::accumulate(values.begin(), values.end(), 0.0);
```

Important: initial value type matters.

```
std::vector<double> xs = {1.5, 2.5};
auto bad = std::accumulate(xs.begin(), xs.end(), 0);   // int accumulation
```

Use `0.0` for double accumulation.

15.5 Erase-remove idiom

```
v.erase(std::remove_if(v.begin(), v.end(), [](int x) {
    return x < 0;
}), v.end());
```

`remove_if` does not shrink the container. It moves kept elements forward and returns the logical end. `erase` removes the tail.

15.6 Lambdas

```
auto greater_than = [](double x, double threshold) {
    return x > threshold;
};
```

Capture by value:

```
double threshold = 100.0;
auto pred = [threshold](double x) {
    return x > threshold;
};
```

Capture by reference:

```
auto pred = [&threshold](double x) {
    return x > threshold;
};
```

Danger: a lambda that outlives a referenced variable will dangle.

15.7 Interview questions

Q: Why prefer STL algorithms?

Answer:

> They express intent clearly, reduce hand-written loop bugs, and are often optimized. They also make code easier to review because the operation is named.

Q: What is erase-remove?

Answer:

> `remove_if` moves elements to keep toward the front and returns a new logical end, but does not change container size. `erase` then removes the unwanted tail. Together they remove elements from sequence containers like vector.

Q: What is a lambda capture?

Answer:

> A lambda capture controls which outside variables are available inside the lambda and whether they are captured by value or reference. Capturing by reference can be dangerous if the lambda outlives the referenced variable.

---

16. Hash tables and probing

16.1 What problem hash tables solve

A hash table provides fast key-value lookup.

Example:

```
std::unordered_map<std::string, double> prices;
prices["AAPL"] = 190.0;
```

Lookup by key is average O(1).

16.2 Hash function

A hash function maps a key to an integer.

```
std::hash<std::string> h;
auto value = h("AAPL");
```

The table maps that hash to a bucket.

16.3 Collision

Two different keys can map to the same bucket. This is a collision.

Collision handling approaches:

1. Separate chaining
2. Open addressing
3. Linear probing
4. Quadratic probing
5. Double hashing

16.4 Separate chaining

Each bucket stores a list/vector of entries.

```
bucket 0: (key1, value1) -> (key2, value2)
bucket 1: empty
bucket 2: (key3, value3)
```

Lookup:

1. Hash key.
2. Go to bucket.
3. Search entries in bucket.

16.5 Open addressing

All entries live in the array. On collision, probe for another slot.

Linear probing:

```
idx = (idx + 1) % capacity;
```

16.6 Load factor

Load factor:

```
number of elements / number of buckets
```

As load factor grows, collisions grow. Rehashing increases capacity.

16.7 Tombstones

For open addressing, deletion needs tombstones.

If you mark a deleted slot as empty, searches may stop too early.

A tombstone says:

> Something was here, so search must continue, but insertion can reuse this slot.

16.8 Interview questions

Q: How does a hash table work?

Answer:

> A hash table uses a hash function to map keys to bucket indices. For lookup, it hashes the key, goes to the bucket, and searches for the key. Collisions are handled by chaining or probing. Average lookup is O(1), but worst-case can be O(n).

Q: What is load factor?

Answer:

> Load factor is the ratio of stored elements to bucket count. Higher load factor means more collisions. Hash tables rehash to maintain efficient operations.

Q: Why are tombstones needed in open addressing?

Answer:

In open addressing, lookup follows a probe chain. If deletion marks a slot empty, the lookup may stop before reaching keys placed later in the chain. Tombstones preserve the chain while allowing reuse.

---

17. Exceptions and error handling

17.1 What exceptions are

Exceptions are a way to signal errors that disrupt normal flow.

```
if (!file) {
    throw std::runtime_error("failed to open file");
}
```

Catching:

```
try {
    run();
} catch (const std::exception& e) {
    std::cerr << e.what() << '\n';
}
```

17.2 RAII makes exceptions safe

If an exception is thrown, stack unwinding destroys local objects.

```
void f() {
    std::vector<int> v(1000);
    std::lock_guard<std::mutex> lock(m);
    throw std::runtime_error("fail");
}
```

`v` is destroyed and lock is released.

17.3 Exception safety guarantees

Basic guarantee

Object remains valid, but state may change.

Strong guarantee

Operation either succeeds or has no effect.

No-throw guarantee

Operation does not throw.

17.4 Copy-and-swap

```
class X {
public:
    X& operator=(X other) {
        swap(other);
        return *this;
    }

    void swap(X& other) noexcept {
        using std::swap;
        swap(data_, other.data_);
    }

private:
    std::vector<int> data_;
};
```

If copying into `other` fails, `this` is unchanged. Swap should be no-throw.

17.5 Exceptions in low-latency code

Some low-latency systems avoid throwing exceptions in hot paths because latency predictability matters.

Alternatives:

```
std::optional<Order> parse_order(std::string_view line);
```

or status codes:

```
enum class ParseError { None, BadPrice, BadQuantity };
```

17.6 Interview questions

Q: Are exceptions bad?

Answer:

> Not inherently. Exceptions work well with RAII and are useful for exceptional failures. But in latency-sensitive hot paths, teams often avoid throwing and use explicit error handling for predictability.

Q: What is the strong exception guarantee?

Answer:

> The operation either completes successfully or leaves the object unchanged, as if the operation never happened.

Q: Why should destructors not throw?

Answer:

> If a destructor throws during stack unwinding from another exception, the program may terminate. Destructors should generally handle cleanup failures internally and not throw.

---

18. Concurrency from novice to intermediate

18.1 Why concurrency is hard

Concurrency is difficult because multiple threads can interleave operations unpredictably.

A line like:

```
++counter;
```

is not one indivisible operation. It can mean:

1. Load counter
2. Add one
3. Store result

Two threads can interleave and lose updates.

18.2 Threads

```
std::thread t([] {
    std::cout << "worker\n";
});

t.join();
```

A `std::thread` must be joined or detached before destruction. Otherwise, program terminates.

C++20 `std::jthread` joins automatically.

18.3 Data race

A data race occurs when:

- Two threads access same memory
- At least one writes
- No synchronization

Data races are undefined behavior.

18.4 Mutex

A mutex protects a critical section.

```
std::mutex m;
int counter = 0;

void increment() {
    std::lock_guard<std::mutex> lock(m);
    ++counter;
}
```

Only one thread can hold the mutex at a time.

18.5 RAII locks

`lock_guard` locks in constructor and unlocks in destructor.

```
{
    std::lock_guard<std::mutex> lock(m);
```

```
    // protected
} // unlock here
```

This prevents forgetting to unlock.

18.6 `unique_lock`

More flexible than lock_guard.

```
std::unique_lock<std::mutex> lock(m);
lock.unlock();
lock.lock();
```

Needed for condition variables.

18.7 Condition variables

Used for waiting until a condition is true.

```
std::mutex m;
std::condition_variable cv;
std::queue<int> q;

void producer(int x) {
    {
        std::lock_guard<std::mutex> lock(m);
        q.push(x);
    }
    cv.notify_one();
}

int consumer() {
    std::unique_lock<std::mutex> lock(m);
    cv.wait(lock, [] { return !q.empty(); });
    int x = q.front();
    q.pop();
    return x;
}
```

Always wait with a predicate because of spurious wakeups.

18.8 Deadlock

Deadlock occurs when threads wait forever for each other.

```
std::mutex a, b;

void t1() {
    std::lock_guard<std::mutex> l1(a);
    std::lock_guard<std::mutex> l2(b);
}

void t2() {
    std::lock_guard<std::mutex> l1(b);
    std::lock_guard<std::mutex> l2(a);
}
```

Prevention:

- Lock in consistent global order.
- Use `std::scoped_lock(a, b)`.
- Keep critical sections small.
- Avoid callbacks while holding locks.

18.9 Reader-writer lock

Multiple readers can read simultaneously. Writer needs exclusive access.

```
std::shared_mutex m;

double read_mid() {
    std::shared_lock lock(m);
    return mid;
}

void update_mid(double x) {
    std::unique_lock lock(m);
    mid = x;
}
```

Use when reads are frequent and writes are less frequent.

18.10 Atomics

Atomics provide safe concurrent access to a single variable.

```
std::atomic<int> counter{0};

void increment() {
    counter.fetch_add(1);
}
```

Use atomics for simple independent variables. Use mutexes for compound state.

18.11 Memory ordering simplified

The safest default is sequential consistency.

```
counter.fetch_add(1); // default seq_cst
```

Relaxed ordering:

```
counter.fetch_add(1, std::memory_order_relaxed);
```

This guarantees atomicity but not ordering with other operations.

Interview-safe answer:

> I use simple mutexes or default atomics unless performance requires weaker memory ordering and I can prove correctness.

18.12 Interview questions

Q: What is a data race?

Answer:

> A data race is concurrent unsynchronized access to the same memory where at least one access is a write. In C++, data races are undefined behavior.

Q: Is `counter++` thread-safe?

Answer:

> Not for a normal integer. It is a read-modify-write sequence and can lose updates. Use a mutex or `std::atomic`.

Q: What is a deadlock?

Answer:

> Deadlock occurs when threads wait on each other in a cycle, so none can proceed. It often happens when locks are acquired in inconsistent order.

Q: How would you implement a reader-writer lock?

Answer:

> In production I would use `std::shared_mutex`. If implementing manually, I would track the number of active readers, whether a writer is active, and possibly waiting writers to avoid starvation, using a mutex and condition variable.

Q: Atomics vs mutexes?

Answer:

> Atomics are good for simple independent variables like counters or flags. Mutexes are better for protecting invariants involving multiple variables because they are easier to reason about.

---

19. Memory layout and performance

19.1 Performance mental model

Performance is often limited by memory access, not arithmetic.

CPU registers are fastest, then cache, then main memory. Accessing contiguous memory is much faster than chasing pointers across memory.

19.2 Cache locality

Vector:

```
std::vector<double> prices;
```

Elements are contiguous. Great for scanning.

List:

```
std::list<double> prices;
```

Nodes may be scattered. Poor cache locality.

19.3 Allocation cost

Heap allocations are relatively expensive and can cause fragmentation/contention.

Bad hot path:

```
void on_tick(const Tick& tick) {
    std::vector<double> tmp;
    tmp.push_back(tick.price);
}
```

Better:

```
class Processor {
public:
    Processor() { tmp_.reserve(1024); }

    void on_tick(const Tick& tick) {
        tmp_.clear();
        tmp_.push_back(tick.price);
    }

private:
    std::vector<double> tmp_;
};
```

Best may be avoiding temporary allocation entirely.

19.4 AoS vs SoA

Array of structs:

```
struct Tick {
    double bid;
    double ask;
    std::int64_t ts;
```

```
};
std::vector<Tick> ticks;
```

Good when processing whole records.

Struct of arrays:

```
struct TickColumns {
    std::vector<double> bids;
    std::vector<double> asks;
    std::vector<std::int64_t> timestamps;
};
```

Good when scanning one column.

19.5 False sharing

False sharing occurs when threads update different variables on the same cache line.

```
struct alignas(64) Counter {
    std::atomic<std::uint64_t> value{0};
};
```

Useful for per-thread counters.

19.6 Profiling

Do not optimize blindly.

Process:

1. Write correct code.
2. Add tests.
3. Profile.
4. Identify bottleneck.
5. Optimize measured bottleneck.
6. Re-measure.

19.7 Interview questions

Q: Why is vector faster than list in many cases?

Answer:

> Vector stores elements contiguously, so iteration benefits from cache locality and prefetching. List requires pointer chasing and separate allocations. Even if list has better asymptotic insertion complexity, vector often wins in real workloads.

Q: What is false sharing?

Answer:

> False sharing happens when different threads update different variables that occupy the same cache line. The cache line bounces between cores, causing performance degradation even though the variables are logically independent.

Q: How do you optimize C++ code?

Answer:

> First ensure correctness, then profile. Common optimizations include reducing allocations, improving data locality, reserving capacity, avoiding unnecessary copies, reducing lock contention, and selecting better algorithms or data structures.

---

20. Numerical C++ for quant roles

20.1 `double` vs `float`

Quant finance generally uses `double` for pricing, risk, and simulation.

Use `float` when:

- Memory bandwidth matters more than precision
- GPU/ML workloads
- Precision requirements are lower

20.2 Floating-point equality

Avoid direct equality for computed values.

```
bool almost_equal(double a, double b) {
    double abs_eps = 1e-12;
    double rel_eps = 1e-12;
    double diff = std::abs(a - b);
    return diff <= abs_eps || diff <= rel_eps * std::max(std::abs(a),
```

```
std::abs(b));
}
```

20.3 NaN and infinity

```
double x = std::numeric_limits<double>::quiet_NaN();
std::isnan(x); // true
```

NaN is not equal to itself.

```
x == x // false if x is NaN
```

20.4 Integer price ticks

Do not use double for order book price levels.

Use integer ticks:

```
using PriceTicks = std::int64_t;
PriceTicks price = 10125; // e.g. cents or ticks
```

Why?

- Exact comparison
- Stable map keys
- No rounding surprises

20.5 Random number generation

Do not use `rand()`.

```
std::mt19937_64 rng(42);
std::normal_distribution<double> normal(0.0, 1.0);

double z = normal(rng);
```

Separate engine from distribution.

20.6 Monte Carlo structure

```
template <typename Payoff>
double monte_carlo(std::size_t n, Payoff payoff) {
    std::mt19937_64 rng(42);
    std::normal_distribution<double> normal(0.0, 1.0);

    double sum = 0.0;
    for (std::size_t i = 0; i < n; ++i) {
        double z = normal(rng);
        sum += payoff(z);
    }
    return sum / static_cast<double>(n);
}
```

20.7 Interview questions

Q: Why use integer ticks for prices?

Answer:

> Order books need exact price-level comparison and equality. Floating-point representation can introduce rounding errors. Integer ticks or scaled integers represent prices exactly and are standard in trading systems.

Q: How do you compare doubles?

Answer:

> For computed floating-point values, use absolute and relative tolerances. Direct equality is usually unsafe except for values that are exactly assigned or used as sentinels.

Q: How do you seed Monte Carlo simulations?

Answer:

> Use deterministic seeds for reproducible tests and research. For production or independent runs, manage seeds carefully to avoid accidental correlation. Each thread should usually have its own RNG engine.

---

21. CSV-to-columnar design

21.1 What this tests

This tests:

- File reading
- Parsing
- Memory layout
- Type conversion
- Error handling
- Performance reasoning

21.2 Row-oriented storage

```
std::vector<std::vector<std::string>> rows;
```

Simple, but inefficient for analytics.

21.3 Columnar storage

```
struct Table {
    std::vector<std::string> names;
    std::vector<std::vector<std::string>> columns;
};
```

Typed columns:

```
using Column = std::variant<
    std::vector<std::int64_t>,
    std::vector<double>,
    std::vector<std::string>
>;

struct TypedTable {
    std::vector<std::string> names;
    std::vector<Column> columns;
};
```

21.4 Why columnar helps

If you only need the `price` column, columnar storage lets you scan only price values contiguously.

This improves:

- Cache locality
- Vectorization potential
- Compression potential
- Analytical query speed

21.5 Parsing edge cases

A real CSV parser must handle:

- Quoted commas
- Escaped quotes
- Empty fields
- Missing fields
- Extra fields
- Type conversion errors
- Very large files
- Different line endings

21.6 Interview answer

Q: How would you convert CSV to columnar format?

Answer:

> I would read the header to identify columns, determine or infer column types, allocate one vector per column, then parse each row and append each value to its corresponding column. For large files, I would process in chunks and avoid storing every field as a temporary string. I would handle quoted fields, missing values, and conversion errors explicitly. Columnar storage is better for analytics because each column is contiguous and can be scanned independently.

Follow-up: How would you make it faster?

Answer:

> Reserve memory when row count is known, parse using `string_view` over a buffer where safe, store numeric values directly in typed vectors, process chunks, avoid per-field allocations, and potentially memory-map the file for large inputs.

---

22. Order book design in C++

22.1 Requirements

An order book supports:

- Add limit order
- Cancel order
- Modify order
- Match incoming order
- Query best bid/ask
- Preserve price-time priority

22.2 Types

```
using OrderId = std::uint64_t;
using Price = std::int64_t;
using Quantity = std::int64_t;

enum class Side { Buy, Sell };
```

Use integer price ticks.

22.3 Basic order

```
struct Order {
    OrderId id;
    Side side;
    Price price;
    Quantity qty;
};
```

22.4 Price levels

```
struct PriceLevel {
    Price price;
    std::list<Order> orders;
    Quantity total_qty = 0;
};
```

Why `list` here?

Normally vector is preferred, but order books often need stable iterators for cancellation. A list can make sense inside price levels.

22.5 Maps for price levels

```
std::map<Price, PriceLevel, std::greater<Price>> bids;
std::map<Price, PriceLevel> asks;
```

- Bids descending: highest bid first.
- Asks ascending: lowest ask first.

22.6 ID lookup for cancellation

```
struct OrderLocation {
    Side side;
    Price price;
    std::list<Order>::iterator it;
};

std::unordered_map<OrderId, OrderLocation> by_id;
```

This lets you find an order quickly by ID.

22.7 Interview answer

Q: How would you design an order book?

Answer:

> I would store bid price levels in a descending ordered map and ask price levels in an ascending ordered map. Each price level stores a FIFO queue of orders to maintain price-time priority. I would also maintain an unordered map from order ID to location for fast cancellation. Prices should be integer ticks, not doubles. For production, I would consider object pools and intrusive lists to reduce allocation overhead.

Follow-up: Why not a heap?

Answer:

> A heap gives fast best-price access, but arbitrary cancellation and modification are awkward. An order book needs fast lookup by order ID and stable management of price levels, so map plus ID lookup is more flexible.

Follow-up: What are the bottlenecks?

Answer:

> Allocations, cache misses from node-based structures, cancellation lookup, and lock contention if multi-threaded. Production systems often use custom memory pools, intrusive lists, and single-thread ownership per book.

---

23. Trading-system design in C++

23.1 Components

A trading system may include:

- Market data handler
- Strategy engine
- Risk engine
- Order manager
- Execution gateway
- Position manager
- Audit log
- Monitoring
- Replay/backtest engine

23.2 State machines

Orders should have explicit states.

```
enum class OrderState {
    Created,
    RiskChecked,
    Sent,
    Acknowledged,
    PartiallyFilled,
    Filled,
    CancelRequested,
    Cancelled,
    Rejected
};
```

Explicit states prevent inconsistent boolean combinations.

23.3 Consistency

Avoid partial updates.

Bad:

```
positions.update(order);
gateway.send(order);
```

If send fails, position state may be wrong.

Better:

- Validate order.
- Reserve risk capacity.
- Send order.
- Update state based on acknowledgments/fills.
- Record events for replay.

23.4 Threading model

Prefer ownership of mutable state by one thread/component when possible.

Example:

- Market data thread receives data.
- Strategy thread processes events.
- Order manager owns order states.
- Components communicate via queues.

This reduces shared mutable state.

23.5 Interview answer

Q: How would you design a trading system?

Answer:

> I would separate market data, strategy, risk, order management, execution, and persistence. Orders would be represented by explicit state machines. Mutable state should have clear ownership, often through event loops or queues rather than shared locks. In C++, I would use RAII for resources, integer price types, bounded queues, minimal allocations in hot paths, and append-only logs for replay and recovery.

Follow-up: How ensure atomicity and consistency?

Answer:

> Use explicit state transitions, avoid partial updates, validate before committing, log important events, and ensure that each state change is owned by one thread or protected by synchronization. External messages should be idempotent where possible.

---

24. Build systems, linking, and ODR

24.1 Compile vs link errors

Compile error:

```
int x = "hello"; // type error
```

Link error:

```
void f();
int main() { f(); }
// f declared but never defined
```

24.2 Include guards

```
#pragma once
```

Prevents multiple inclusion of the same header.

24.3 One Definition Rule

Do not define non-inline globals in headers.

Bad:

```
// config.h
int global_counter = 0;
```

Good:

```
// config.h
extern int global_counter;
```

```
// config.cpp
int global_counter = 0;
```

24.4 Interview questions

Q: What is the One Definition Rule?

Answer:

> The One Definition Rule says that entities like functions and variables should have exactly one definition in the program, with exceptions for templates and inline functions. Violating it can cause linker errors or undefined behavior.

Q: Why define templates in headers?

Answer:

> The compiler needs the template definition available when instantiating it for a specific type. Therefore, template definitions are usually placed in headers.

---

25. Testing and debugging C++

25.1 Why testing C++ is different

C++ can have memory bugs that do not appear immediately. You need tests plus tools.

25.2 Sanitizers

AddressSanitizer:

```
-fsanitize=address -g -fno-omit-frame-pointer
```

Finds:

- Use-after-free
- Out-of-bounds access
- Some stack bugs

UndefinedBehaviorSanitizer:

```
-fsanitize=undefined -g
```

ThreadSanitizer:

```
-fsanitize=thread -g
```

Finds data races.

25.3 Invariant testing

For order book:

- Best bid is highest bid.
- Best ask is lowest ask.
- ID map matches actual orders.
- Quantities are consistent.

For hash table:

- Inserted keys are findable.
- Deleted keys are absent.
- Rehash preserves entries.

For concurrency:

- No lost updates.
- No data races.
- No deadlocks.

25.4 Interview questions

Q: How would you test an order book?

Answer:

> I would test simple add/cancel/match cases, edge cases like empty book and partial fills, and invariants after every operation. I would add randomized tests against a simple reference model and replay tests from recorded events. I would also run sanitizers to catch memory and threading bugs.

Q: How do you find memory bugs?

Answer:

> Use AddressSanitizer, UndefinedBehaviorSanitizer, and careful ownership design. Prefer RAII and standard containers to avoid manual memory management bugs.

---

26. High-yield interview question bank with expanded answers

26.1 Core language

Q: What is stack vs heap?

Answer:

> Stack usually refers to automatic storage for local variables whose lifetime is tied to scope. Heap refers to dynamic storage allocated at runtime. Stack allocation is fast and automatically cleaned up. Heap allocation is flexible but has allocation overhead and requires ownership management. In modern C++, heap memory should usually be owned by containers or smart pointers.

Follow-up:

> Is a vector stored on stack or heap?

Answer:

> The vector object itself can be on the stack if local, but its elements are usually stored in heap memory owned by the vector.

Q: What is RAII?

Answer:

> RAII means Resource Acquisition Is Initialization. A resource is acquired in a constructor and released in a destructor. It gives automatic cleanup and exception safety. Examples are `vector`, `unique_ptr`, `lock_guard`, and file streams.

Q: What is undefined behavior?

Answer:

> Undefined behavior means the C++ standard gives no meaning to the program. The compiler may assume it never occurs. Examples include out-of-bounds access, null dereference, dangling references, data races, and signed overflow.

26.2 Ownership

Q: How do you decide between value, reference, pointer, and smart pointer?

Answer:

> If the object is small or ownership should be local, use value. For read-only large objects, use `const&`. For required mutation, use non-const reference. For optional non-owning access, use pointer. For exclusive ownership, use `unique_ptr`. For shared lifetime ownership, use `shared_ptr`, but only when necessary.

Q: Why avoid shared_ptr by default?

Answer:

> `shared_ptr` adds reference-count overhead, can create cycles, and makes ownership less clear. `unique_ptr` or value ownership is usually simpler and better unless shared ownership is truly required.

26.3 Move semantics

Q: What is a moved-from object?

Answer:

> A moved-from object is still valid and destructible, but its value is unspecified unless documented. You can assign a new value to it or destroy it, but should not rely on its previous contents.

Q: Why return by value?

Answer:

> Return by value is usually efficient in modern C++ due to copy elision and move semantics. It also avoids dangling references and gives clear ownership to the caller.

26.4 STL

Q: Why is vector default container?

Answer:

> It is simple, contiguous, cache-friendly, supports random access, and has amortized O(1) append. In many real workloads, its memory locality beats node-based containers.

Q: When use list?

Answer:

> When you need stable iterators and frequent O(1) insertion/removal given an iterator. Even then, consider whether deque, vector with indices, or another design may be better.

26.5 Templates

Q: Why can templates cause code bloat?

Answer:

> The compiler may instantiate separate code for each type used with a template. This can increase binary size, especially for large template functions used with many types.

26.6 Concurrency

Q: What is spurious wakeup?

Answer:

> A condition variable wait may wake up even if no notification occurred or the condition is not true. Therefore, always wait with a predicate or loop.

Q: Why avoid holding locks while calling unknown code?

Answer:

> Unknown code may try to acquire another lock, call back into your object, block for a long time, or throw. This can cause deadlocks, latency spikes, or inconsistent state.

26.7 Performance

Q: Why profile before optimizing?

Answer:

> Human intuition about bottlenecks is often wrong. Profiling identifies the actual hot path. Optimizing non-bottlenecks adds complexity without improving performance.

---

27. Coding drills to reach intermediate level

Drill 1: RAII timer

Implement a class:

```
class Timer {
public:
    explicit Timer(std::string name);
    ~Timer();
};
```

It should print elapsed time in destructor.

Concepts:

- Constructor
- Destructor
- `std::chrono`
- Move/copy decisions

Drill 2: File wrapper

Implement a wrapper around `FILE*` that:

- Opens in constructor
- Closes in destructor
- Is non-copyable
- Is movable

Concepts:

- RAII
- Rule of Five
- Move semantics

Drill 3: Vector invalidation demo

Write code showing that references into a vector can dangle after reallocation. Then fix it with `reserve` or indices.

Drill 4: Linear probing hash map

Implement:

- insert
- find
- erase
- rehash

Explain tombstones.

Drill 5: Thread-safe queue

Implement:

- push
- blocking pop
- shutdown

Use mutex and condition variable.

Drill 6: Market data cache

Use `shared_mutex` to allow many readers and one writer.

Drill 7: Mini order book

Implement:

- add order
- cancel order
- best bid/ask
- match market order

Use map + unordered_map.

Drill 8: Monte Carlo simulator

Implement:

- payoff as lambda/template
- deterministic seed
- standard error
- optional parallel version

---

28. Study plan: novice to intermediate

Phase 1: Absolute foundations

Learn:

- Basic syntax
- Functions
- Classes
- References and pointers
- `const`
- `vector`, `string`, `map`, `unordered_map`

Goal:

> Be able to write simple programs without manual memory management.

Phase 2: Ownership and lifetime

Learn:

- RAII
- Smart pointers
- Rule of Zero/Five
- Dangling references
- Undefined behavior

Goal:

> Be able to explain who owns every object in your code.

Phase 3: Intermediate C++

Learn:

- Move semantics
- Templates
- Iterators
- STL algorithms
- Exception safety
- Virtual functions

Goal:

> Be able to answer common C++ interview questions precisely.

Phase 4: Systems and quant C++

Learn:

- Concurrency
- Cache locality
- Numeric issues
- Order books
- Market data caches
- CSV parsers
- Trading state machines

Goal:

> Be able to design and discuss realistic quant systems in C++.

---

29. Final readiness checklist

You are intermediate interview-ready when you can explain and code:

Language

- Stack vs heap
- Object lifetime
- Undefined behavior
- RAII
- Rule of Zero/Five
- Constructors/destructors
- `const`
- References and pointers
- Smart pointers
- Copy/move semantics

STL

- `vector`, `deque`, `list`
- `map`, `unordered_map`
- `optional`, `variant`
- `span`, `string_view`
- Iterators and invalidation
- Algorithms and lambdas

OOP/templates

- Virtual functions
- Vtables
- Virtual destructors
- Object slicing
- Templates
- `typename`
- Type traits
- Concepts

Concurrency

- Threads
- Mutexes
- RAII locks
- Deadlocks
- Condition variables
- Reader-writer locks
- Atomics
- Data races

Quant systems

- Floating-point issues
- Integer price ticks
- Monte Carlo structure
- CSV-to-columnar design
- Order book design
- Trading-system state machines
- Performance profiling

---

30. C++26 direction: coroutines, async, and `std::execution`

You are right that this topic was missing. For modern C++, especially C++26-oriented C++, you should understand both:

1. **Coroutines**: a language feature for writing suspendable functions.
2. **Asynchronous execution models**: library abstractions for scheduling, composing, and running asynchronous work.

Important version distinction:

- **Coroutines** entered the language in C++20.
- `std::execution` **/ senders-receivers** are the major C++26 direction for standard asynchronous execution.
- C++26 support will vary by compiler and standard library for some time, so in real code you may see libraries such as Boost.Asio, Folly, libunifex, or stdexec-style implementations before full standard-library support is mature.

The mental model you need for interviews is not just syntax. You should understand why the old `std::async` / `std::future` model is limited, what coroutines solve, what they do not solve, and why C++26 introduces sender/receiver-based execution.

30.1 Synchronous vs concurrent vs parallel vs asynchronous

These words are often confused.

Synchronous

Synchronous code runs one step after another. The caller waits until the operation finishes.

```
auto data = read_file("prices.csv");
auto result = compute(data);
```

`compute` starts only after `read_file` finishes.

Concurrent

Concurrent means multiple tasks are in progress during the same time period. They may or may not be literally running at the same instant.

Example:

- One task waits for network data.
- Another task processes previous data.

Parallel

Parallel means multiple tasks are literally executing at the same time on different cores or execution units.

Example:

```
// Conceptually parallel work
compute_risk_for_book_1();
compute_risk_for_book_2();
```

If these run on two CPU cores simultaneously, they are parallel.

Asynchronous

Asynchronous means the caller starts an operation and does not block waiting for it immediately. The result arrives later through a callback, future, coroutine continuation, receiver, or event loop.

Example idea:

```
start_read(socket, callback_when_done);
```

The program can do other work while the read is pending.

Interview answer

> Synchronous code blocks until work completes. Concurrent code has multiple tasks in progress. Parallel code executes tasks simultaneously, usually on multiple cores. Asynchronous code starts work and receives completion later without necessarily blocking the caller. Async is about non-blocking structure; parallelism is about simultaneous execution.

30.2 Old C++ async: `std::async`, `std::future`, and `std::promise`

Before C++26 `std::execution`, standard C++ had `std::async`, `std::future`, and `std::promise`.

`std::async`

```
#include <future>
#include <iostream>

int compute() {
    return 42;
}

int main() {
    std::future<int> fut = std::async(std::launch::async, compute);

    // do other work
```

```
    int result = fut.get(); // blocks until result is ready
    std::cout << result << '
';
}
```

What happens?

- `std::async` starts a task.
- It returns a `std::future<int>`.
- `future.get()` waits for the task and returns the result.

Problem: launch policy

```
std::async(compute);
```

Without explicit policy, implementation may choose async or deferred execution.

Use:

```
std::async(std::launch::async, compute);
```

if you really want asynchronous execution.

`std::future` limitation

`std::future` is useful but limited:

- Usually one-shot result retrieval.
- Poor composition.
- No built-in continuation chaining in standard C++.
- Cancellation is not naturally integrated.
- Execution location is not always explicit.
- It does not form a general async graph abstraction.

This is one reason C++26 `std::execution` matters.

`std::promise`

A promise lets one part of code set a result while another waits through a future.

```
#include <future>
#include <thread>
#include <iostream>
```

```
int main() {
    std::promise<int> promise;
    std::future<int> future = promise.get_future();

    std::thread worker([p = std::move(promise)]() mutable {
        p.set_value(123);
    });

    std::cout << future.get() << '
';
    worker.join();
}
```

Interview answer

> `std::async` and `std::future` are older standard tools for launching work and retrieving results. They are simple but not very composable. Futures represent one eventual result, but standard futures do not naturally express complex async pipelines, cancellation, execution resources, or continuation graphs. C++26 `std::execution` addresses these limitations with senders, receivers, and schedulers.

30.3 What is a coroutine?

A coroutine is a function that can suspend and later resume execution.

A function becomes a coroutine if it contains one of:

```
co_await
co_yield
co_return
```

Mental model:

> A normal function has one entry and runs until it returns. A coroutine can pause in the middle, return control to its caller, and later continue from where it paused.

Coroutines are useful for:

- Async I/O
- Lazy generators
- Pipelines
- Event-driven systems
- State machines that would otherwise require callbacks

Normal function

```
int f() {
    return 42;
}
```

Runs to completion.

Coroutine-like idea

```
task<int> f() {
    co_await something_async();
    co_return 42;
}
```

This can suspend while waiting.

30.4 Coroutines are stackless

C++ coroutines are **stackless**.

That means when a coroutine suspends, it does not keep an entire thread stack alive. Instead, the compiler stores the coroutine state in a **coroutine frame**.

The coroutine frame contains things like:

- Local variables that must survive suspension
- Current suspension point
- Promise object
- Bookkeeping needed to resume/destroy the coroutine

Example:

```
task<int> example() {
    int x = 10;
    co_await some_async_operation();
    co_return x + 1;
}
```

`x` must survive across the `co_await`, so it lives in the coroutine frame.

Interview answer

> C++ coroutines are stackless. When they suspend, they return control to the caller and store resumable state in a coroutine frame rather than preserving an entire call stack. This makes them useful for scalable async code, but the coroutine return type and promise type determine how suspension, resumption, and result delivery work.

30.5 The three coroutine keywords

`co_return`

Completes a coroutine and returns a value through the coroutine's promise machinery.

```
task<int> compute_async() {
    co_return 42;
}
```

Do not use plain `return` in a coroutine.

`co_yield`

Produces a value and suspends.

Used for generators.

```
generator<int> count_up() {
    for (int i = 0; ; ++i) {
        co_yield i;
    }
}
```

This can lazily produce values one at a time.

`co_await`

Suspends until an awaitable operation completes.

```
task<void> handle_socket(Socket& socket) {
    auto msg = co_await socket.async_read();
```

```
    co_await socket.async_write(msg);
}
```

This looks sequential but can be non-blocking.

Interview answer

> `co_return` completes a coroutine, `co_yield` produces a value and suspends, and `co_await` suspends until an awaitable operation is ready. The coroutine's return type defines the promise and await behavior.

30.6 Coroutine return types: why `task<int>` is not built into basic C++

A beginner surprise:

```
int f() {
    co_return 42; // invalid
}
```

A coroutine must return a type that satisfies coroutine requirements. The return type controls the coroutine behavior through a nested `promise_type`.

Examples of coroutine return-type concepts:

```
task<int>       // async result
generator<int>  // lazy sequence
```

The C++ language provides coroutine machinery, but it does not magically provide every high-level async type you might want. Libraries define types like `task<T>`, `generator<T>`, or awaitable socket operations.

Interview answer

> Coroutines are a language mechanism, not a complete async runtime by themselves. The coroutine return type defines how results, suspension, resumption, and destruction work. That is why libraries provide types like `task<T>` or `generator<T>`.

30.7 Coroutine frame, promise, awaiter: the intermediate mental model

When the compiler sees a coroutine, it transforms it into a state machine.

Important pieces:

1. **Coroutine frame**: stores state needed across suspension.
2. **Promise object**: controls result production and coroutine behavior.
3. **Coroutine handle**: low-level handle used to resume or destroy the coroutine.
4. **Awaiter**: object that defines how `co_await` behaves.

Awaiter protocol

An awaiter can define:

```
bool await_ready();
void await_suspend(std::coroutine_handle<> h);
auto await_resume();
```

Meaning:

- `await_ready()` says whether suspension is needed.
- `await_suspend()` runs when coroutine suspends.
- `await_resume()` gives the result when coroutine resumes.

Very simple conceptual awaiter:

```
#include <coroutine>

struct AlwaysSuspend {
    bool await_ready() const noexcept {
        return false;
    }

    void await_suspend(std::coroutine_handle<>) const noexcept {
        // control returns to caller
    }

    void await_resume() const noexcept {
        // no result
    }
};
```

Interview answer

> A coroutine is lowered into a state machine. Its frame stores state across suspensions. The promise object defines how the coroutine returns values and handles completion. `co_await` uses an awaiter with `await_ready`, `await_suspend`, and `await_resume` to decide whether to suspend and what value to produce when resumed.

30.8 Coroutines vs threads

Coroutines are not threads.

Thread

A thread has its own execution stack and may run in parallel with other threads.

Coroutine

A coroutine is a suspendable computation. It only runs when resumed by some thread.

A coroutine can make async code look sequential, but it does not automatically create parallelism.

```
task<void> f() {
    co_await async_read();
    co_await async_write();
}
```

This code can be non-blocking, but some event loop or scheduler must resume it.

Interview answer

> A coroutine is not a thread. A coroutine is a suspendable function/state machine. It runs on a thread when resumed, but suspension does not block that thread. Threads provide execution resources; coroutines structure asynchronous control flow.

30.9 Coroutines vs callbacks

Callback style:

```
socket.async_read([](Message msg) {
    process(msg);
    socket.async_write(response, [] {
        // done
```

```
    });
});
```

Coroutine style:

```
task<void> handle(Socket& socket) {
    Message msg = co_await socket.async_read();
    process(msg);
    co_await socket.async_write(response);
}
```

Coroutines make async code look sequential, which improves readability and error handling.

Interview answer

> Coroutines help avoid deeply nested callback code. They let asynchronous operations be written in direct style while still allowing suspension and resumption under the hood.

30.10 Coroutines vs `std::future`

`std::future` represents a result that becomes available later.

Coroutines are a way to write suspendable functions.

They solve different layers of the problem.

A coroutine may return a future-like type, but standard `std::future` is not a great coroutine building block because it lacks continuation/composition features.

Interview answer

> `std::future` is a one-shot result holder. A coroutine is a language feature for suspension and resumption. Coroutines can produce future-like objects, but the quality of the async model depends on the return type and scheduler. C++26 `std::execution` gives a more composable model than plain futures.

30.11 C++26 `std::execution`: why it matters

C++26 introduces a standard execution-control model through `std::execution`.

The key abstractions are:

1. **Scheduler**
2. **Sender**
3. **Receiver**
4. **Operation state**

This model is also called **senders/receivers**.

The purpose is to provide a standard vocabulary for asynchronous and parallel work.

Why not just use `std::thread`?

`std::thread` is low-level. It gives you a thread but not a composable async pipeline.

Why not just use `std::future`?

Standard futures are hard to compose, do not naturally express cancellation, and do not clearly control where continuations execute.

Why `std::execution`?

It aims to make async work:

- Composable
- Generic over execution resources
- Explicit about where work runs
- Able to model success, error, and cancellation
- Suitable for CPUs, thread pools, event loops, GPUs, and other execution resources

Interview answer

> C++26 `std::execution` provides a standard model for asynchronous execution based on schedulers, senders, receivers, and operation states. It improves on older futures by making async work composable, explicit about execution context, and structured around value, error, and cancellation channels.

30.12 Scheduler

A scheduler is a lightweight handle to an execution resource.

Execution resources could include:

- Thread pool
- Event loop
- GPU stream

- IO context
- Custom low-latency executor

Conceptually:

```
auto sched = pool.get_scheduler();
```

A scheduler can produce a sender that represents work scheduled on that resource.

Interview answer

> A scheduler represents where work can run. It is a handle to an execution resource such as a thread pool, event loop, or GPU stream. In sender/receiver terminology, a scheduler can create senders that complete on that resource.

30.13 Sender

A sender describes asynchronous work.

Important: a sender is often **lazy**. Creating a sender usually does not start work.

Conceptual example:

```
auto s = execution::just(42);
```

This represents an operation that will complete with value `42` when connected and started.

A sender can complete through one of three channels:

1. Value/success
2. Error/failure
3. Stopped/canceled

Interview answer

> A sender is a description of asynchronous work. It does not necessarily run immediately. It sends completion to a receiver through value, error, or stopped channels.

30.14 Receiver

A receiver consumes the completion of a sender.

It handles:

- Successful values
- Errors
- Cancellation/stopped signal

Conceptually:

```
struct MyReceiver {
    void set_value(int x);
    void set_error(std::exception_ptr e);
    void set_stopped();
};
```

The exact standard APIs are more formal, but the mental model is:

> Receiver is a generalized callback object with separate channels for success, error, and cancellation.

Interview answer

> A receiver is the consumer side of an async operation. It receives completion signals: value for success, error for failure, and stopped for cancellation.

30.15 Operation state and `start`

A sender and receiver are connected to form an operation state.

Conceptually:

```
auto op = execution::connect(sender, receiver);
execution::start(op);
```

Key point:

> Work does not begin until the operation state is started.

The operation state contains the state needed by the asynchronous operation and must remain alive until the operation completes.

Interview answer

> Connecting a sender and receiver produces an operation state. Starting the operation state begins the work. The operation state owns or contains the state needed for the asynchronous operation and must remain alive until completion.

30.16 Sender composition

The power of senders is composition.

Conceptually, you can build pipelines:

```
auto work = just(10)
          | then([](int x) { return x + 1; })
          | then([](int x) { return x * 2; });
```

This describes a graph of work.

For quant systems, this could represent:

```
read market data
→ parse message
→ normalize symbol
→ update book
→ run strategy
→ send order candidate to risk
```

The value is that execution context, error propagation, and cancellation can be modeled explicitly.

Interview answer

> Sender algorithms let us compose async operations into pipelines or graphs. Instead of manually nesting callbacks or blocking on futures, we can describe the work and how completions flow through value, error, and stopped channels.

30.17 `run_loop`

C++26 includes `std::execution::run_loop`, an execution context with a manually driven event loop.

Mental model:

```
queue work
run loop pulls work
execute completions
```

This matters because async work needs an execution resource. A run loop is one such resource.

In production quant systems, you may have custom event loops:

- One loop per instrument group
- One loop per gateway connection
- One loop for market data handling
- One loop for risk or persistence

Interview answer

> A run loop is an execution context that owns a queue of work and executes it when driven. It is useful for event-driven systems and gives explicit control over where asynchronous completions run.

30.18 Coroutines and `std::execution` together

Coroutines and senders/receivers are complementary.

- Coroutines make asynchronous code look sequential.
- Senders/receivers provide a composable execution model.

A library can allow:

```
task<void> strategy_loop() {
    auto tick = co_await next_tick_sender;
    co_await risk_check_sender;
}
```

The coroutine uses `co_await`, while the underlying async operations may be represented as senders.

Interview answer

> Coroutines are language-level syntax for suspension and resumption. `std::execution` is a library model for composing and scheduling asynchronous work. They can work together: senders can be made awaitable, and coroutines can provide a direct-style interface over sender-based async operations.

30.19 Async in quant systems

Where async matters in quant C++:

1. **Market data ingestion**
2. Non-blocking socket reads
3. Parsing messages as they arrive
4. Avoiding blocked strategy loops
5. **Order gateway**
6. Send order
7. Await acknowledgment
8. Handle cancel/replace/fill asynchronously
9. **Risk checks**
10. Parallel or async pre-trade checks
11. Avoid blocking market data processing
12. **Backtesting**
13. Async file reads
14. Pipeline from disk → parser → simulator → metrics
15. **Research systems**
16. Parallel parameter sweeps
17. Distributed simulation tasks
18. Async result collection
19. **Logging and persistence**
20. Queue logs asynchronously
21. Avoid blocking hot path on disk I/O

Example design: async market data pipeline

```
socket read sender
→ parse sender
→ normalize sender
→ book update sender
→ strategy sender
→ risk sender
→ order send sender
```

Each stage can have explicit execution context and error handling.

Interview answer

> In quant systems, async is useful whenever waiting would block critical work: network I/O, file I/O, order acknowledgments, logging, and distributed computation. The key is to avoid blocking hot paths while maintaining explicit ownership, ordering, and backpressure.

30.20 Backpressure

Async systems can fail if producers are faster than consumers.

Example:

- Market data arrives faster than strategy can process.
- Log messages arrive faster than disk can write.
- Risk tasks pile up in a thread pool.

Backpressure means the system has a way to slow producers, drop data intentionally, batch work, or reject work.

Strategies:

- Bounded queues
- Dropping stale updates
- Coalescing updates
- Applying flow control
- Prioritizing important messages
- Measuring queue depth

Interview answer

> Backpressure prevents an async system from accumulating unbounded work. In trading systems, bounded queues and explicit overload policies are essential because unbounded latency can be worse than dropping or coalescing low-priority work.

30.21 Cancellation

In async systems, cancellation is not necessarily an error.

Example:

- Cancel a stale risk calculation.
- Stop a strategy loop.
- Cancel an order operation after disconnect.
- Stop reading a file when backtest is aborted.

C++26 senders/receivers explicitly model a stopped channel separate from value and error.

Interview answer

> Cancellation should be modeled explicitly. It is not always an error. A well-designed async system distinguishes success, failure, and cancellation so cleanup and state transitions are correct.

30.22 Error handling in async code

Synchronous errors:

```
try {
    auto x = compute();
} catch (...) {
}
```

Async errors must be transported to the completion handler, future, receiver, or coroutine result.

In sender/receiver thinking:

- Success → value channel
- Failure → error channel
- Cancellation → stopped channel

Interview answer

> Async error handling requires errors to be propagated through the async completion mechanism. In C++26 sender/receiver style, errors are separate from cancellation and success, which makes pipeline behavior more explicit.

30.23 Thread pools

A thread pool owns a set of worker threads and a queue of tasks.

Basic mental model:

```
submit task → queue → worker thread picks task → task runs
```

Why use a thread pool instead of creating threads repeatedly?

- Thread creation is expensive.
- Too many threads cause context-switch overhead.
- A pool can bound concurrency.
- A pool can manage scheduling policies.

Basic thread pool interface concept

```
class ThreadPool {
public:
    template <typename F>
    void submit(F&& f);
};
```

In C++26-oriented design, a thread pool would expose a scheduler compatible with `std::execution`.

Interview answer

> A thread pool amortizes thread creation cost and bounds concurrency. In a C++26-style design, a thread pool is an execution resource, and its scheduler can produce senders for work that should run on the pool.

30.24 Important pitfalls in async C++

Pitfall 1: dangling references across async boundaries

Bad:

```
void submit_work(ThreadPool& pool) {
    std::string symbol = "AAPL";
    pool.submit([&] {
        use(symbol); // dangling if runs after submit_work returns
```

```
    });
}
```

Fix:

```
void submit_work(ThreadPool& pool) {
    std::string symbol = "AAPL";
    pool.submit([symbol = std::move(symbol)] {
        use(symbol);
    });
}
```

Capture by value when work outlives the current scope.

Pitfall 2: blocking inside async tasks

If every worker blocks waiting for more work from the same pool, you can deadlock or starve the pool.

Bad pattern:

```
auto f = pool.submit(task_a);
auto result = f.get(); // worker blocks waiting
```

Better:

- Compose continuations.
- Use non-blocking async chains.
- Use separate execution resources when blocking is unavoidable.

Pitfall 3: unbounded queues

Unbounded queues can hide overload until latency explodes.

Use bounded queues and explicit policies.

Pitfall 4: unclear execution context

You should know where continuation code runs.

Question to ask:

> Which thread or scheduler executes this callback/receiver/coroutine continuation?

Pitfall 5: exceptions disappearing

Async errors must be captured and delivered somewhere.

30.25 Interview questions: coroutines

Q: What is a coroutine?

Answer:

> A coroutine is a function that can suspend execution and resume later. In C++, a function becomes a coroutine if it uses `co_await`, `co_yield`, or `co_return`. Coroutines are compiled into state machines with coroutine frames storing state across suspension points.

Follow-up: Is a coroutine a thread?

Answer:

> No. A coroutine is not an execution resource. It runs on a thread when resumed, but suspension does not block that thread. Threads provide execution; coroutines provide suspendable control flow.

Follow-up: What is stored in a coroutine frame?

Answer:

> State needed to resume the coroutine, including local variables that live across suspension points, the current suspension point, the promise object, and bookkeeping for resumption and destruction.

Q: What is `co_await`?

Answer:

> `co_await` suspends a coroutine until an awaitable operation is ready. The awaiter controls whether suspension happens, what happens at suspension, and what value is produced on resume through `await_ready`, `await_suspend`, and `await_resume`.

Q: What is `co_yield`?

Answer:

> `co_yield` produces a value from a coroutine and suspends, commonly used to implement lazy generators.

Q: What is `co_return`?

Answer:

> `co_return` completes a coroutine and passes the result to the coroutine promise. Coroutines do not use ordinary `return` for returning values.

Q: Why are coroutines useful for async I/O?

Answer:

> They let asynchronous code be written in a sequential style without blocking threads. Instead of nested callbacks, you write `co_await read`, then process, then `co_await write`.

Q: What are coroutine pitfalls?

Answer:

> Dangling references across suspension, unclear lifetime of coroutine frames, forgetting who resumes/destroys the coroutine, blocking inside coroutines, and assuming coroutines automatically create parallelism.

30.26 Interview questions: async and C++26 `std::execution`

Q: What problem does C++26 `std::execution` solve?

Answer:

> It provides a standard vocabulary and framework for asynchronous and parallel execution. Older tools like `std::thread` and `std::future` are low-level or poorly composable. `std::execution` introduces schedulers, senders, receivers, and operation states to express where work runs, how it composes, and how success, error, and cancellation are delivered.

Q: What is a sender?

Answer:

> A sender describes asynchronous work. It can complete with a value, an error, or a stopped signal. It is typically lazy: creating a sender describes work but does not necessarily start it.

Q: What is a receiver?

Answer:

> A receiver consumes the completion of a sender. It handles separate channels for success, error, and cancellation.

Q: What is a scheduler?

Answer:

> A scheduler is a lightweight handle to an execution resource such as a thread pool, event loop, or GPU stream. It controls where work runs.

Q: What is an operation state?

Answer:

> Connecting a sender and receiver creates an operation state. Starting the operation state begins the work. The operation state must remain alive until the async operation completes.

Q: Why are senders/receivers better than futures?

Answer:

> They are more composable, make execution context explicit, model cancellation separately from errors, and can represent async work across many execution resources. Standard futures are mostly one-shot result holders and do not naturally form async graphs.

Q: Does `std::execution` require coroutines?

Answer:

> No. Senders/receivers and coroutines are separate but complementary. `std::execution` provides an async composition and scheduling model. Coroutines provide language syntax for suspension and resumption. Libraries can integrate them.

Q: Will C++26 automatically give me a production thread pool?

Answer:

> Not necessarily in the way many developers expect. The standard execution model provides abstractions for execution resources and composition. Actual thread-pool implementations and maturity will depend on standard library support and ecosystem libraries.

30.27 Coding drills for coroutines and async

Drill 1: Explain coroutine transformation

Take this conceptual coroutine:

```
task<int> f() {
    int x = 10;
    co_await something();
    co_return x + 1;
}
```

Explain:

- Why `x` must survive suspension
- What the coroutine frame stores
- Why `task<int>` must provide coroutine machinery
- Who resumes the coroutine

Drill 2: Generator mental model

Implement or study a simple `generator<int>` that uses:

```
co_yield
```

Explain how each value is produced lazily.

Drill 3: Async lifetime bug

Given:

```
void f(ThreadPool& pool) {
    std::string s = "AAPL";
    pool.submit([&] { use(s); });
}
```

Explain the bug and fix it.

Drill 4: Future vs sender design

Design a three-stage pipeline:

```
load file → parse rows → compute metrics
```

Describe how it would look with:

- blocking synchronous code
- futures
- coroutine style
- sender/receiver style

Drill 5: Async order gateway

Design an async order flow:

```
send order → await ack → process fills → handle cancel
```

Explain:

- Where state lives
- How cancellation works
- How errors propagate
- What must not block

30.28 How much coroutine detail do quant interviews expect?

For many quant C++ interviews, you do not need to implement a full coroutine type from scratch. But for intermediate C++26-oriented confidence, you should be able to explain:

- What makes a function a coroutine
- Difference between coroutine and thread
- `co_await`, `co_yield`, `co_return`
- Coroutine frame and lifetime
- Promise/awaiter at a high level
- Why coroutines help async I/O
- Why they do not automatically give parallelism
- How they relate to event loops/schedulers
- Why C++26 `std::execution` matters
- Sender, receiver, scheduler, operation state
- Value/error/stopped channels
- Common async lifetime bugs

That is enough for most interviews unless the role specifically focuses on coroutine libraries, networking frameworks, or C++ standard-library internals.

---

31. C++26 features you should know for quant interviews

C++26 is broader than async. For quant interviews, you do not need every feature, but you should know the high-impact direction.

31.1 `std::execution`

Most relevant for modern async, parallel execution, and composable task graphs.

Know:

- Scheduler
- Sender
- Receiver
- Operation state
- Value/error/stopped channels
- `run_loop`
- Relationship to coroutines

31.2 `std::inplace_vector`

A fixed-capacity vector with dynamic size.

Mental model:

```
vector-like API
fixed maximum capacity
storage is inside the object
no heap allocation after construction
```

Why quant developers care:

- Useful in low-latency hot paths.
- Avoids heap allocation.
- Keeps contiguous storage.
- Good when maximum size is known.

Example use case:

```
small list of order modifications
small batch of risk checks
fixed maximum number of legs in a strategy
```

Interview answer:

> `inplace_vector` is useful when I want vector-like contiguous storage but with fixed capacity and no dynamic allocation. That can be valuable in low-latency code where heap allocations are undesirable.

31.3 `std::hive`

Formerly based on colony-like containers.

Mental model:

- Sequence container
- Stable references/iterators under many operations
- Reuses erased element memory
- Not contiguous like vector

Why care:

- Useful when stable references matter and frequent insert/erase occurs.
- Potential alternative to list-like structures in some systems.

Interview answer:

> `hive` is useful for workloads with frequent insertion and erasure where pointer/reference stability matters. It is not a replacement for vector in cache-critical scanning, but it can be better than list-like structures for some dynamic object sets.

31.4 `std::linalg`

C++26 adds standard linear algebra interfaces.

Why quant researchers care:

- Matrix/vector operations
- Numerical code standardization
- Potential BLAS-style backend integration

Interview angle:

> For serious quant numerical work, I would still evaluate Eigen, BLAS/LAPACK, or vendor libraries, but `std::linalg` shows that standard C++ is moving toward more native numerical abstractions.

31.5 `std::simd`

SIMD means single instruction, multiple data.

Useful for:

- Vectorized numerical loops
- Monte Carlo paths
- Risk calculations
- Signal processing

Mental model:

```
operate on multiple numbers at once using vector registers
```

Interview answer:

> SIMD can speed up data-parallel numerical loops by applying one instruction to multiple values. For quant code, it may help Monte Carlo, pricing grids, and risk calculations, but data layout and memory bandwidth still matter.

31.6 Hazard pointers and RCU

C++26 adds standard-library support related to safe memory reclamation patterns such as hazard pointers and RCU.

Why this matters:

- Lock-free and read-mostly concurrent data structures need safe memory reclamation.
- You cannot simply delete nodes while another thread might still read them.

Interview answer:

> In lock-free or read-mostly data structures, memory reclamation is hard because another thread may still hold a pointer to a removed node. Hazard pointers and RCU are techniques for safely delaying reclamation until readers are done.

31.7 Contracts

C++26 includes contracts direction through contract assertions.

Mental model:

```
state preconditions, postconditions, and assertions more explicitly
```

Quant relevance:

- Risk limits
- Order invariants
- Matrix dimension requirements
- Valid price/quantity assumptions

Interview answer:

> Contracts are useful for documenting and checking assumptions such as valid order quantities, non-null dependencies, or matrix dimension compatibility. They help make invariants explicit.

31.8 Reflection

C++26 moves toward compile-time reflection.

Why care:

- Serialization
- Schema generation
- Logging
- Data binding
- Config systems

Quant example:

```
reflect fields of MarketDataMessage
→ generate parser/logging/schema code
```

Interview answer:

> Reflection can reduce boilerplate for serialization, logging, schema generation, and generic tooling. For quant systems with many message types, compile-time reflection could eventually simplify safe, efficient infrastructure.

---

32. Updated C++26 interview-readiness checklist

To be C++26-oriented intermediate, add these to your checklist:

Coroutines

- What makes a function a coroutine
- `co_await`, `co_yield`, `co_return`
- Coroutine frame
- Promise type
- Awaiter protocol
- Coroutine vs thread
- Coroutine vs callback
- Coroutine lifetime pitfalls

Async and execution

- `std::async` / `future` limitations
- Thread pool mental model
- Scheduler
- Sender
- Receiver
- Operation state
- Value/error/stopped channels
- Lazy async work
- Backpressure
- Cancellation
- Event loop / `run_loop`

C++26 library direction

- `std::execution`
- `std::inplace_vector`
- `std::hive`
- `std::simd`
- `std::linalg`
- Hazard pointers / RCU
- Contracts
- Reflection

---

33. Final advice

To move from novice to intermediate, focus less on memorizing syntax and more on repeatedly asking:

1. Who owns this object?
2. How long does it live?
3. Can this pointer/reference dangle?
4. Can this container invalidate my iterator?

5. Is this copy expensive?
6. Can this move be used safely?
7. Is this code thread-safe?
8. Is this data layout cache-friendly?
9. Is this API communicating ownership clearly?
10. Can I explain this tradeoff to an interviewer?

If you can answer these questions while writing code, you are no longer a C++ beginner. You are developing the reasoning style expected from an intermediate C++ engineer in quant interviews.